Intelligence Artificielle

Pr. A.CHOUKRI

ali.choukri@uit.ac.ma

Faculté des sciences, Kénitra

2025/2026

L'intelligence artificielle

Qu'est-ce que c'est ?

Qu'est-ce que intelligence artificielle ?

Définition :

L'Intelligence Artificielle (IA) consiste à mettre en œuvre un certain nombre de techniques visant à permettre aux machines d'imiter une forme d'intelligence réelle.

Qu'est-ce que intelligence artificielle ?

La notion voit le jour dans les années 1950 grâce au mathématicien Alan Turing.

Il se posait la question qui allait bouleverser le monde : **Les machine peuvent-elles faire preuve de signes d'intelligence humaine ?**

$\Rightarrow$ Il décrit un test connu actuellement sous le nom **Test de Turing**

Test de Turing

C'est un test dans lequel un humain interagit à l'aveugle avec un autre humain, puis avec une machine programmée pour formuler des réponses sensées.

Si le sujet n'est pas capable de faire la différence, alors la machine a réussi le test et, selon Alan Turing, peut être considérée comme "*intelligente*".

L'IA n'est pas une science fiction, c'est une réalité

L'IA se fixe comme but la modélisation de l'intelligence prise comme phénomène, de même que la physique, la chimie ou la biologie qui ont pour but de modéliser d'autres phénomènes.

Types de l'IA

L'IA faible

C'est un programme informatique qui peut **aider** aux activités mentales humaines.

L'IA forte ou généralisée

Elle désigne tout système capable d'apprendre et d'effectuer n'importe quelle tâche qu'un humain serait capable de faire.

Ce sont des machines qui **pensent**, avec une **conscience** de soi et qui ont des **sentiments**.

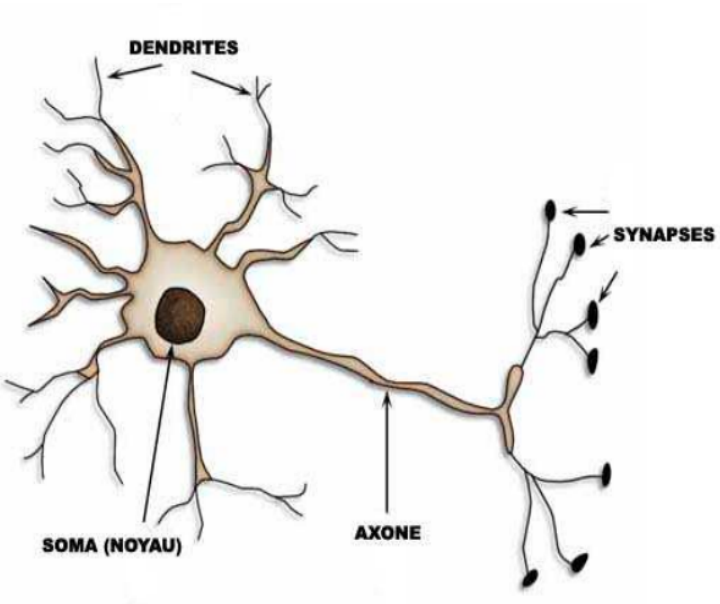
À quel stade en est l'IA forte ?

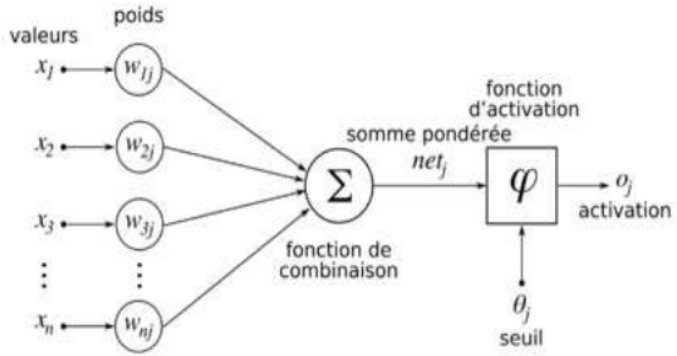
L'IA forte est encore loin de devenir une réalité.

- Les machines les plus intelligentes échouent complètement lorsqu'on leur demande d'exécuter de nouvelles tâches.

L'IA en quelques dates

- 1940 : La cybernétique : *comme science du fonctionnement de l'esprit humain*. (**Mathématicien Norbert Wiener**).
- 1943 : Premier modèle mathématique du neurone biologique, le neurone formel (**Neurologues Warren McCulloch, et Walter Pitts**)





L'IA en quelques dates

- 1949 : Invention la règle qui permet de doter les neurones formels de capacités d'apprentissage (Neuropsychologue **Donald Hebb**)
- 1950 : **Test Turing**
- 1956 : Naissance de terme **Intelligence Artificielle** **John McCarthy**.
- 1960 : Rosenblatt's Perceptron, premier ordinateur à utiliser un réseau de neurones permettant à la machine d'apprendre en fonction de ses réussites et ses échecs

L'IA en quelques date

- 1997 : Victoire de Deep Blue sur Kasparov
- 2009 : La voiture autonome de Google
- 2011 : Le programme Watson d'IBM gagne au Jeopardy
- 2012 : Deep learning
- 2016 : Le programme AlphaGo de Google gagne en jeu de GO.

L'IA : Pourquoi maintenant ?

L'IA : Pourquoi maintenant ?

Trois facteurs expliquent le nouvel essor de l'IA aux alentours de 2010 :

1. Big Data : Réaliser des algorithmes de classification nécessite de grandes masses de données chose qui n'était pas possible à l'époque.
2. Capacités de calculs avec la nouvelle génération des processeurs et des cartes graphiques.
3. Réduction des coûts financiers.

Domaines de l'IA

L'intelligence artificielle s'est déjà glissée dans notre vie :

- Nos voitures,
- Nos téléphones,
- Nos ordinateurs,
- Nos armes de défense
- Nos systèmes de transport.
- La santé
- etc.

Programmation classique vs IA

Les machines ne sont plus programmées ; elles apprennent.

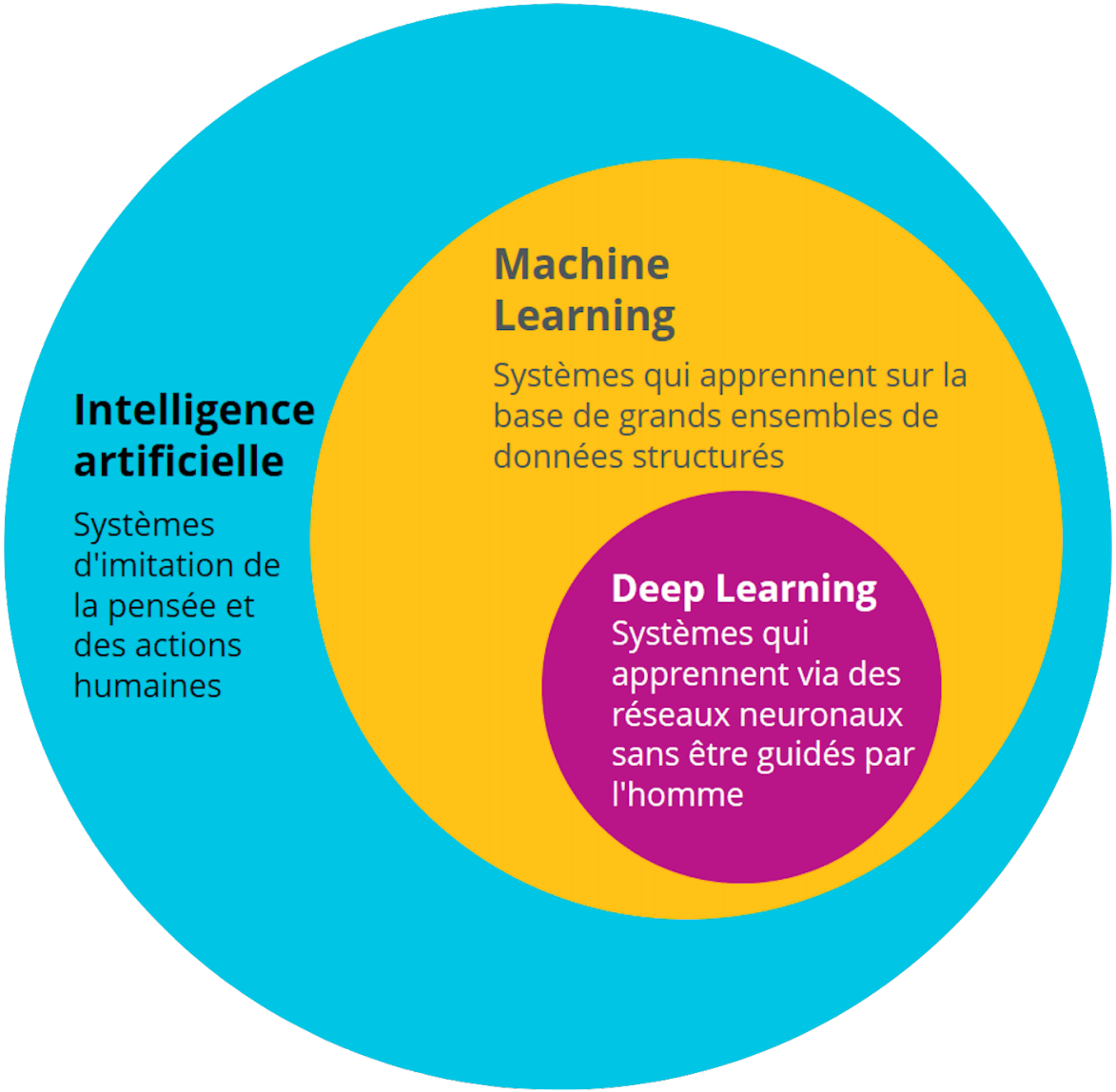
Il ne s'agit plus de coder les règles explicitement comme auparavant, mais de laisser les machines les découvrir seules sur la base d'une quantité massive de données.

De l'IA au Machine Learning

- Au paravant, l'IA n'a pas toujours considéré l'apprentissage comme essentiel à l'intelligence.
- Depuis quelques années, on associe presque toujours l'intelligence aux capacités d'apprentissage, grâce auquel un système intelligent :
  - Peut améliorer ses performances avec l'expérience.
  - Peut apprendre à exécuter de nouvelles tâches.
  - Peut acquérir de nouvelles compétences.

Naissance du Machine Learning.

Les sous-domaines de l'IA



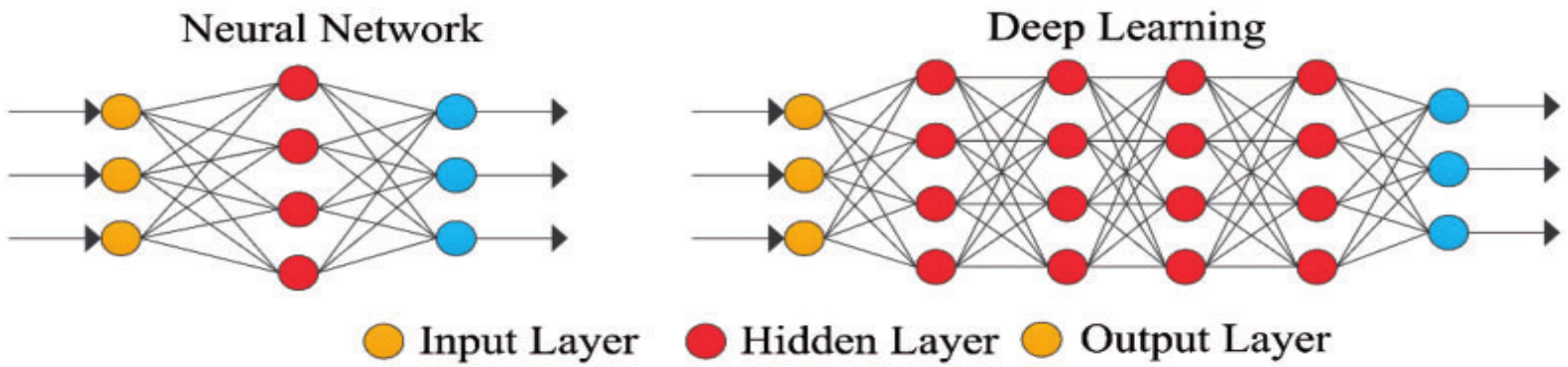
Machine learning

Machine learning

- L'apprentissage automatique, (en anglais : machine learning), est un champ d'étude de l'intelligence artificielle qui se fonde sur des approches mathématiques et statistiques pour donner aux ordinateurs la capacité d'"apprendre" à partir de données.

Deep learning (apprentissage profond)

- Le deep learning est un type d'intelligence artificielle dérivé du machine learning où la machine est capable d'apprendre par elle-même.
- Il s'appuie sur un réseau de neurones artificiels s'inspirant du cerveau humain. Ce réseau peut être composé de dizaines voire de centaines de "couches" de neurones chacune recevant et interprétant les informations de la couche précédente.



Machine Learning

Types d'apprentissage ?

Pour donner à un ordinateur la capacité d'apprendre, on utilise des méthodes d'apprentissage inspirées de la façon dont nous, les êtres humains, apprennent à faire des choses.

Parmi ces méthodes :

- **L'apprentissage supervisé** (Supervised Learning)
- **L'apprentissage non supervisé** (Unsupervised Learning)
- **L'apprentissage par renforcement** (Reinforcement Learning)

L'**apprentissage supervisé** est lorsque l'on fournit à une machine beaucoup d'exemples qu'elle doit étudier.

Quatre notions à connaitre en apprentissage supervisé :

1. **Le Dataset :** L'Algorithme d'apprentissage démarre d'un Dataset qui contient nos données.
2. **Le modèle et ses paramètres :** A partir de ce Dataset, on crée un modèle, qui n'est autre qu'une fonction mathématique dont les coefficients sont les paramètres du modèle.
3. **La Fonction Coût :** C'est l'ensemble des erreurs donné par le modèle.
4. **L'Algorithme d'apprentissage :** Laisser la machine trouver quels sont les paramètres du modèle qui minimisent la *Fonction Coût*.

Dataset :

L'ensemble des exemples (x, y) sont regroupés dans un tableau que l'on appelle Dataset :

- La variable y porte le nom de target (la cible). C'est la valeur que l'on cherche à prédire.
- La variable x porte le nom de feature (facteur). Un facteur influence la valeur de y.

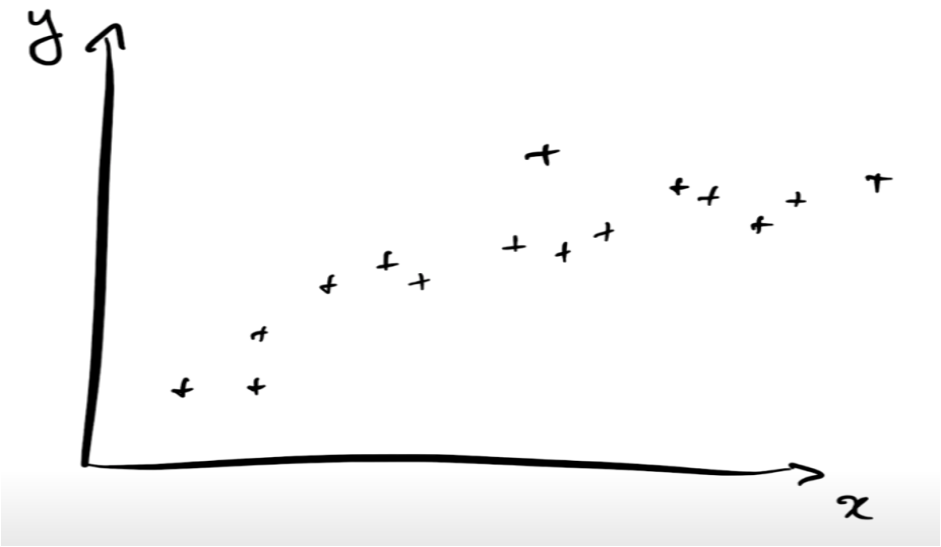
Exemple : Ci-dessous, un dataset qui regroupe des exemples d'appartements avec leur prix y ainsi que certaines de leurs caractéristiques x_i (features).

Target y	Features x_1	x_2	x_3
Prix	**Surface m2**	**N chambres**	**Qualité**
€ 313,000.00	124	3	1.5
€ 2,384,000.00	339	5	2.5
€ 342,000.00	179	3	2
€ 420,000.00	186	3	2.25
€ 550,000.00	180	4	2.5
€ 490,000.00	82	2	1
€ 335,000.00	125	2	2

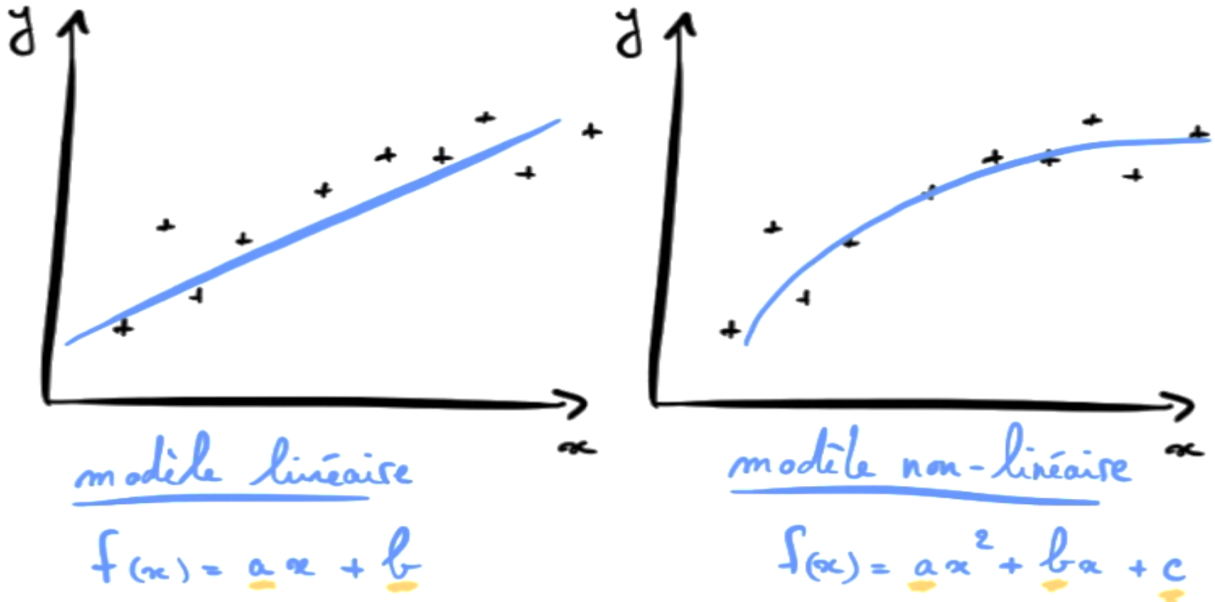
	x	y
1		
2	313	75
3	784	89
4	892	26
5	803	75
6	752	83
7	781	23
8	153	29
9	536	95
10	313	75
11	784	89
12	892	26
13	803	75
14	752	83
15	781	23
16	153	29
17	536	95
18	313	75
19	784	89
20	892	26
21	803	75
22	752	83
23	781	23
24	153	29
25	536	95

Supposons que nous voulons que la machine apprenne la relation qui relie x à y.

Supposons que notre dataset donne le nuage de point suivant :



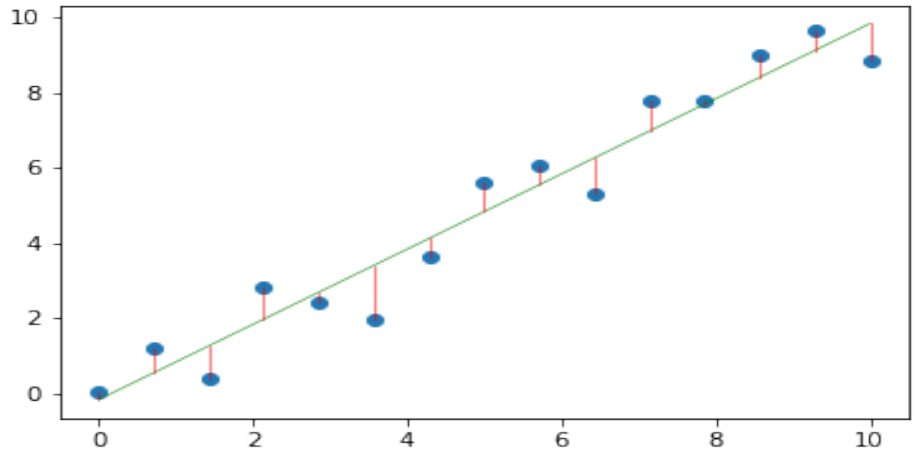
♦ **Développer un modèle à partir du Dataset :** Nous pouvons tracer le modèle linaire (à gauche) ou polynomiale (à droite)



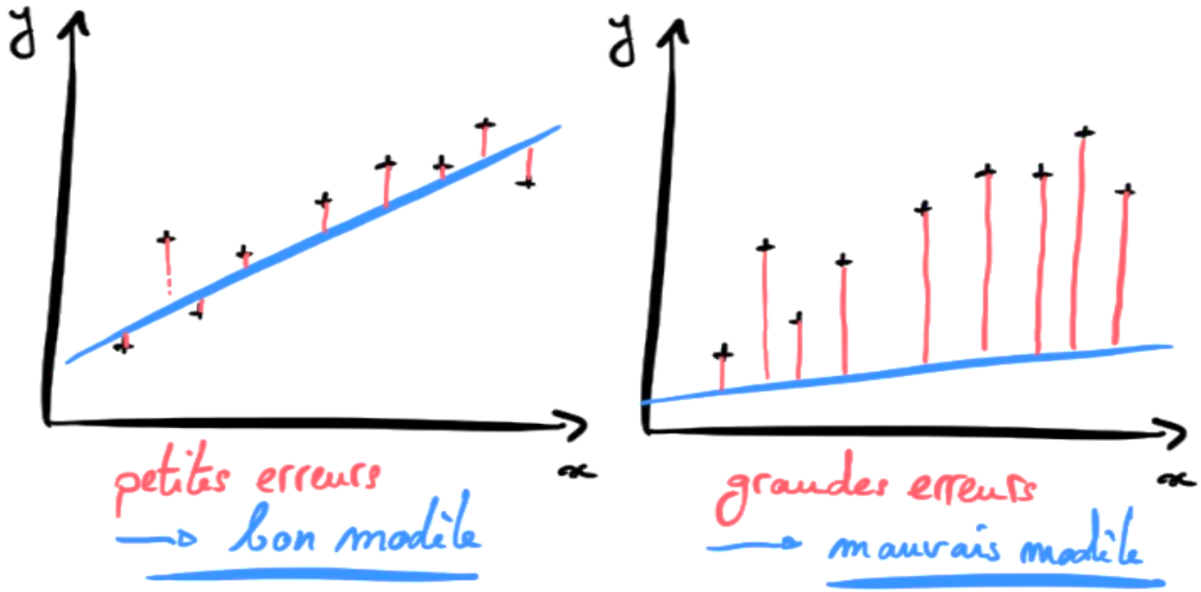
⇒ On définit a, b, c, etc. comme étant les paramètres d'un modèle.

♦ Les erreurs du modèle - la Fonction Coût

Un modèle retourne toujours des erreurs par rapport au Dataset. On appelle Fonction Coût l'ensemble de ces erreurs (le plus souvent on prend la moyenne quadratique des erreurs).



Un bon modèle, c'est un modèle qui donne de petites erreurs, donc une petite Fonction Coût.



♦ **Entrainer le modèle ⇒ Minimiser la Fonction Coût**

L'objectif central en Supervised Learning, c'est de trouver les paramètres du modèle qui minimisent la Fonction Coût.

⇒ **Algorithme d'apprentissage.**

L'exemple le plus courant étant l'algorithme de Gradient Descent.

♦ **Les applications du Supervised Learning**

Avec le Supervised Learning on peut développer des modèles pour résoudre deux types de problèmes :

- Les problèmes de **Régression**
- Les problèmes de **Classification**

♦ **La Régression :**

Dans la régression, on cherche à prédire la valeur d'une variable continue, c'est-à-dire une variable qui peut prendre une infinité de valeurs.

Exemple :

◊ Prédire le prix d'un appartement (y) selon sa surface habitable (x_1), localisation (x_2) sa qualité (x_3) sa proximité avec un parc (x_4) etc.

◊ Prédire la quantité d'essence consommée (y) selon la distance parcourue (x_1), la vitesse (x_2), l'état du véhicule (x_3), etc.

♦ La Classification :

Dans la classification, on cherche à classer un objet dans différentes classes, c'est-à-dire que l'on cherche à prédire la valeur d'une variable discrète (qui ne prend qu'un nombre fini de valeurs).

Exemple :

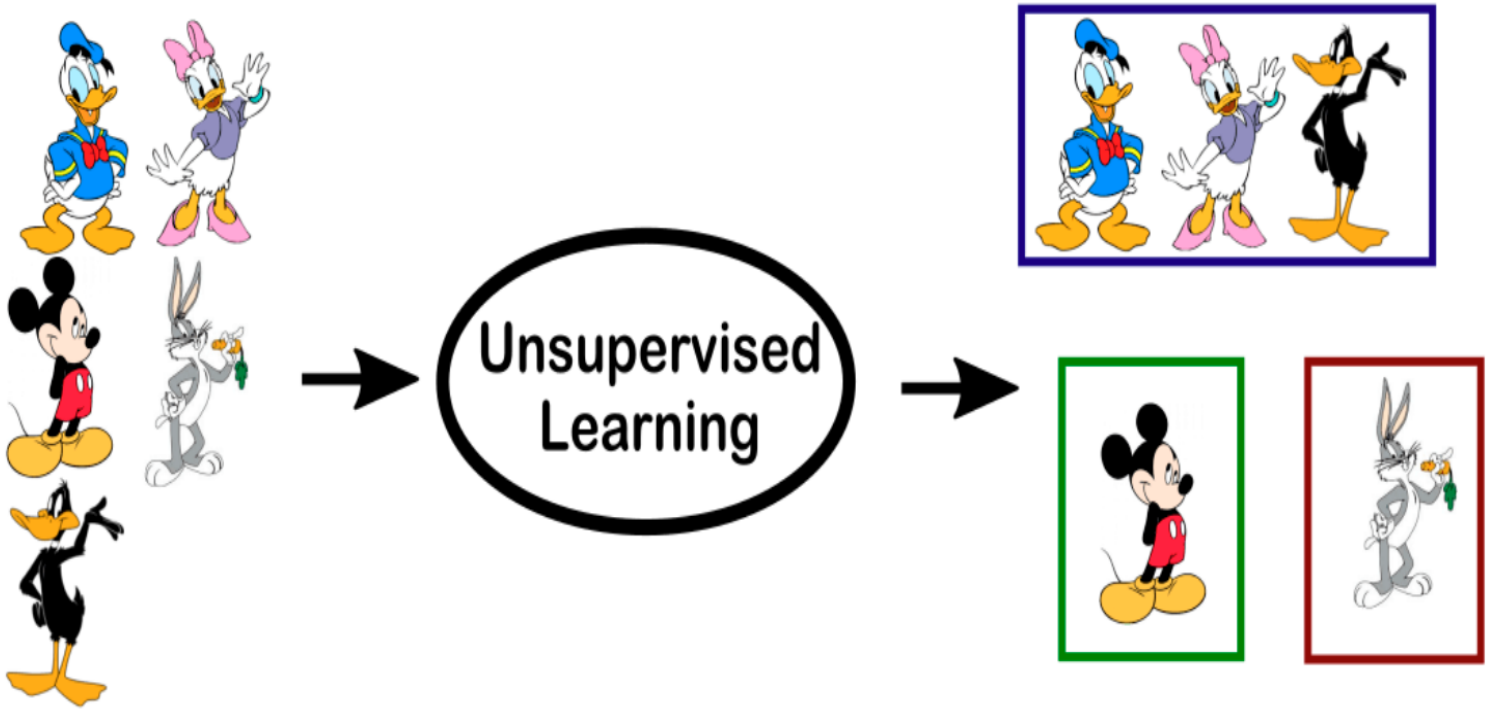
◊ Prédire si un email est un spam (classe $y = 1$) ou non (classe $y = 0$) selon le nombre de liens présent dans l'email (x_1) et le nombre de fautes d'orthographe (x_2), etc.

◊ Prédire si une tumeur est maligne ($y = 1$) ou bénigne ($y = 0$) selon la taille de la tumeur (x_1), l'âge du patient (x_2), les antécédent dans la famille (x_3), etc

L'apprentissage non supervisé est principalement utilisé dans la segmentation (clustering), pour regrouper un ensemble d'éléments hétérogènes sous forme de sous groupes homogènes ou liés par des caractéristiques communes.
La qualité d'une méthode de classification est mesurée par sa capacité à :

- Découvrir certains ou tous les motifs cachés.
- Maximiser l'homogénéité des données au sein des groupes
- Former des groupes aussi distincts que possible

Exemple : Regrouper des images.

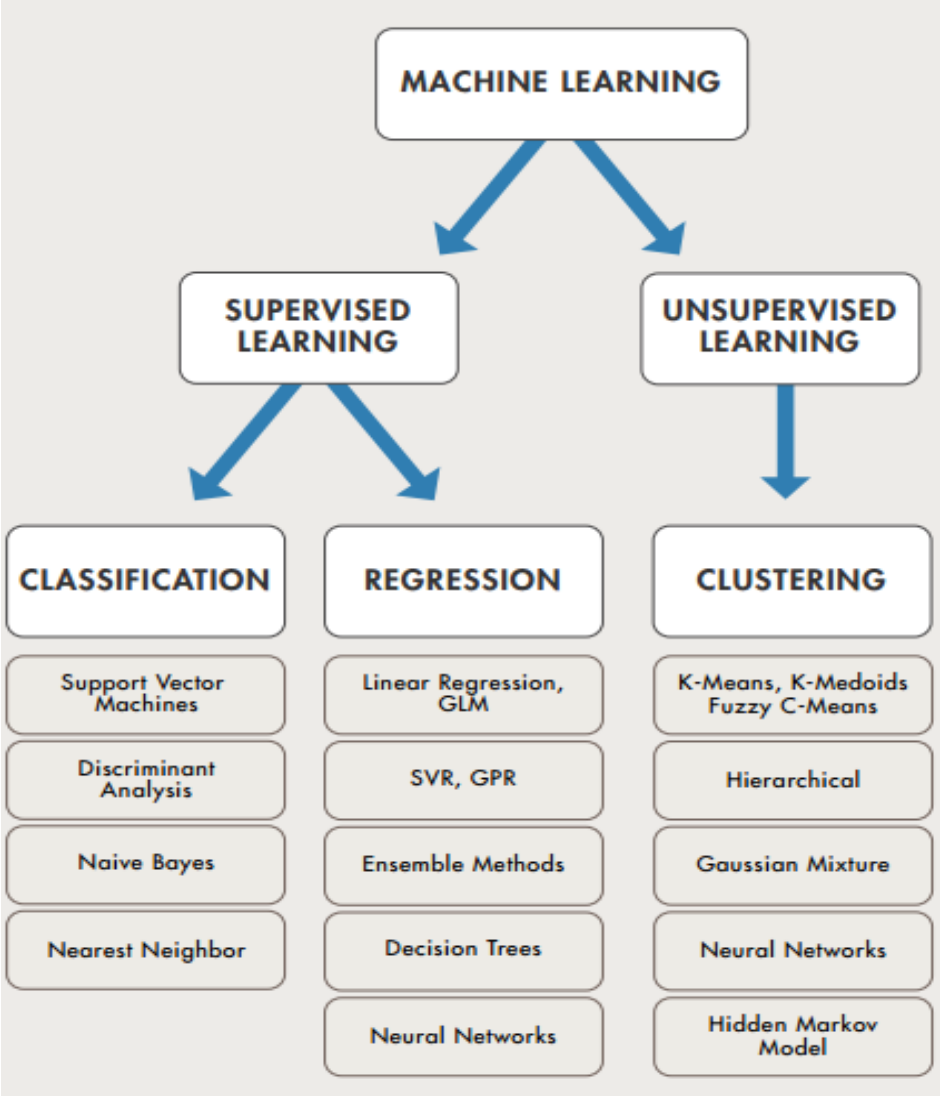


- **L'apprentissage supervisé vs L'apprentissage non supervisé**
  - **L'apprentissage supervisé :** classer un nouvel individu parmi un ensemble de classes prédéfinies : on connaît les classes a priori
  - **L'apprentissage non supervisé :** le nombre et la définition des classes ne sont pas donnés a priori

- Cette dernière méthode s'inspire de la façon dont nous éduquons nos animaux de compagnie, en leur offrant une friandise quand ils font une bonne action.
- Elle consiste , en général, à laisser les ordinateurs apprendre de leurs expériences grâce à un système de récompense ou de pénalité.
- Pour y parvenir, l'ordinateur essaye toutes les façons possibles et apprend de ses erreurs tout en essayant de trouver la méthode qui lui permet de maximiser sa récompense.

Machine Learning

Quelques algorithmes



L'apprentissage supervisé

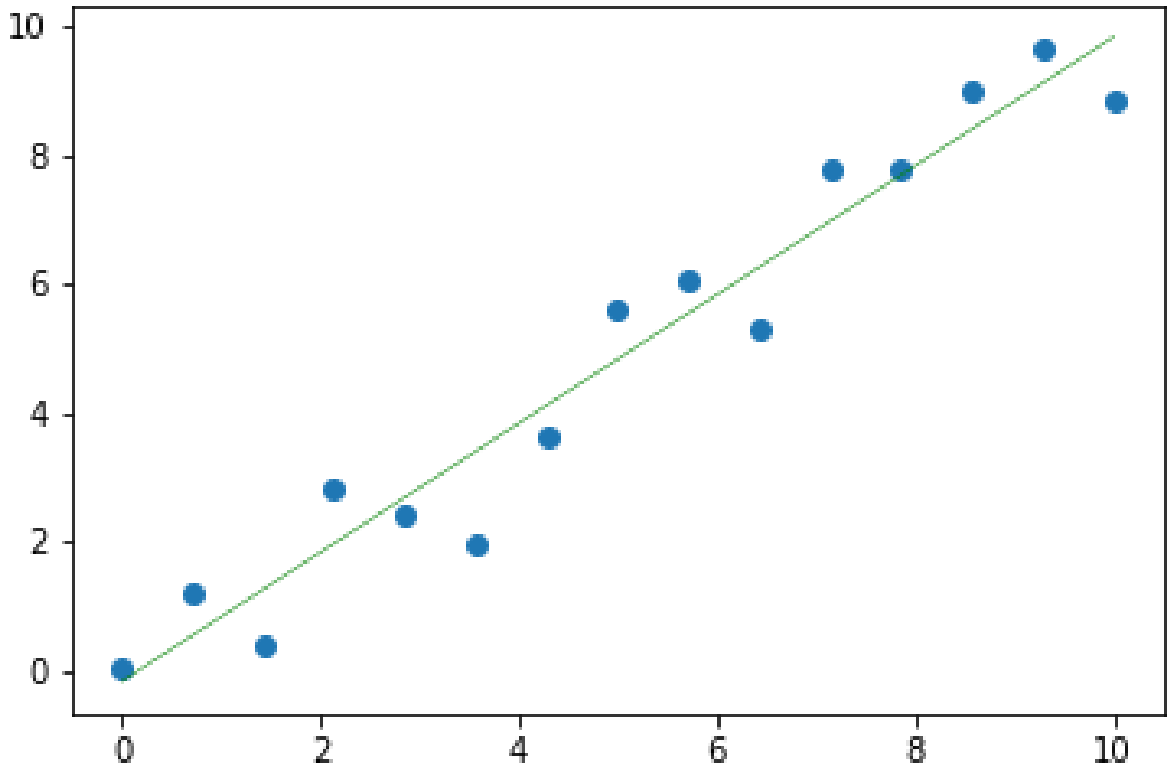
La régression linéaire

LA RÉGRESSION LINÉAIRE

LE MODELE

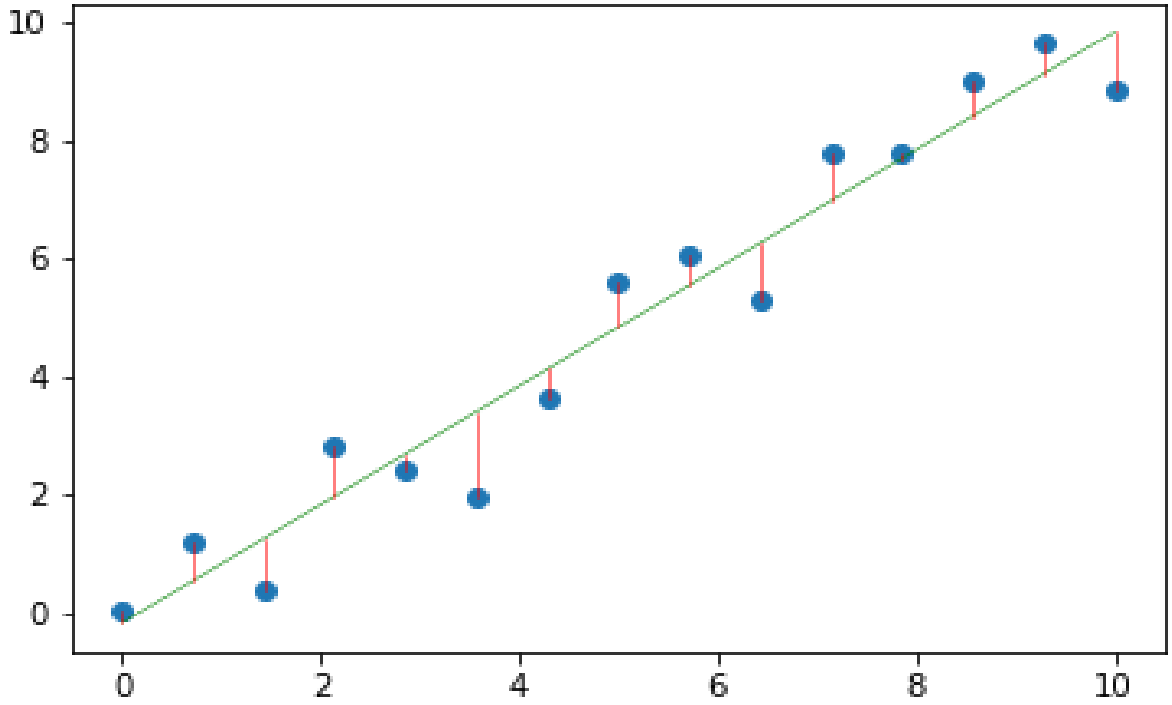
Le régression linéaire consiste à trouver les paramètres (a, b) de la fonction :

$$f(x) = ax + b$$

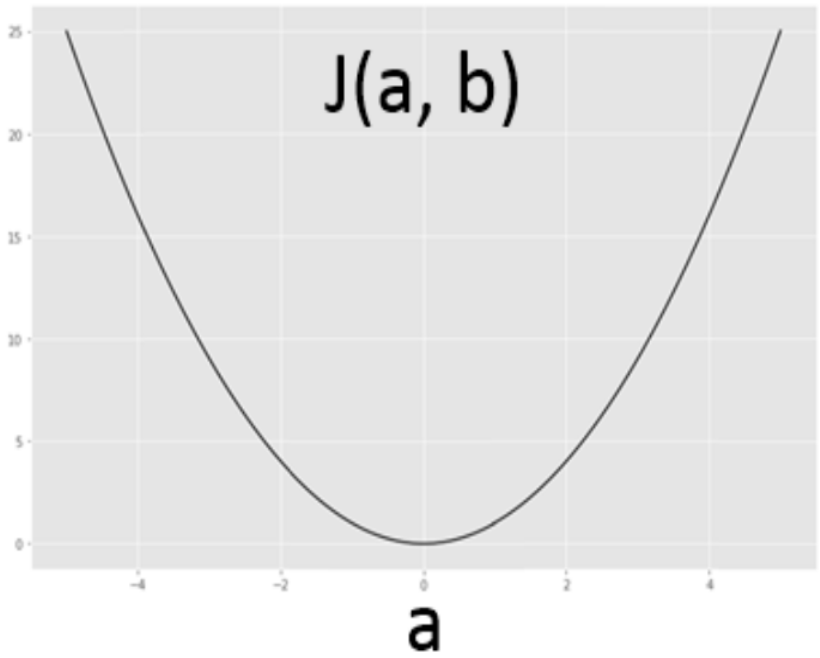


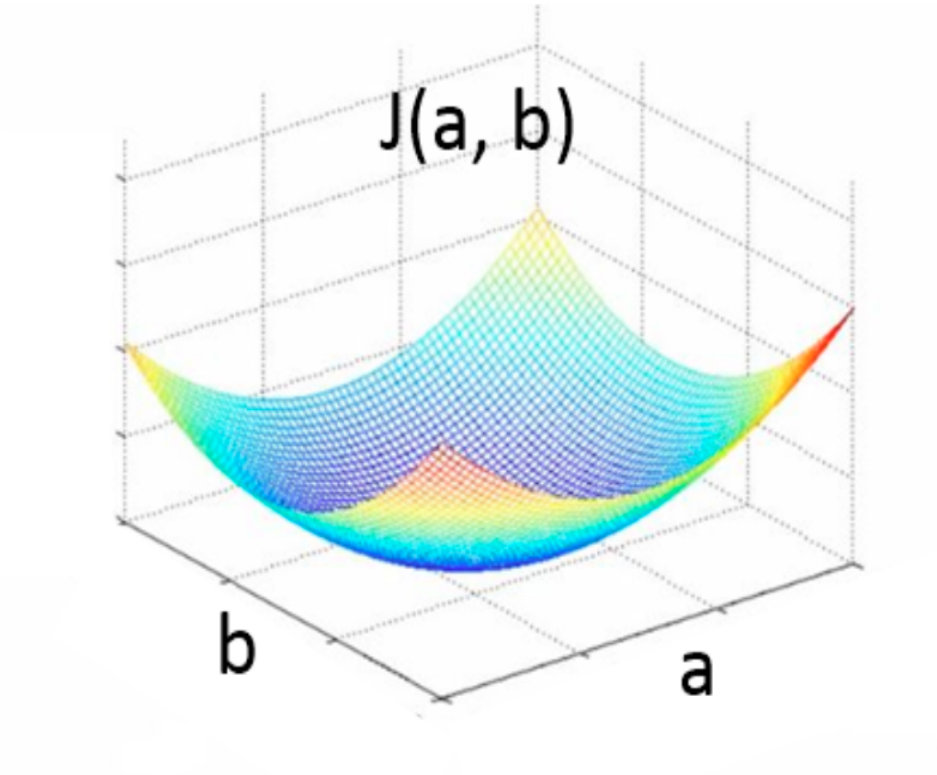
La fonction coût (l'erreur quadratique moyenne) est définie par :

$$J(a,b) = \frac{1}{m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$



L'algorithme d'apprentissage consiste à minimiser la fonction coût dont l'allure est la suivante :





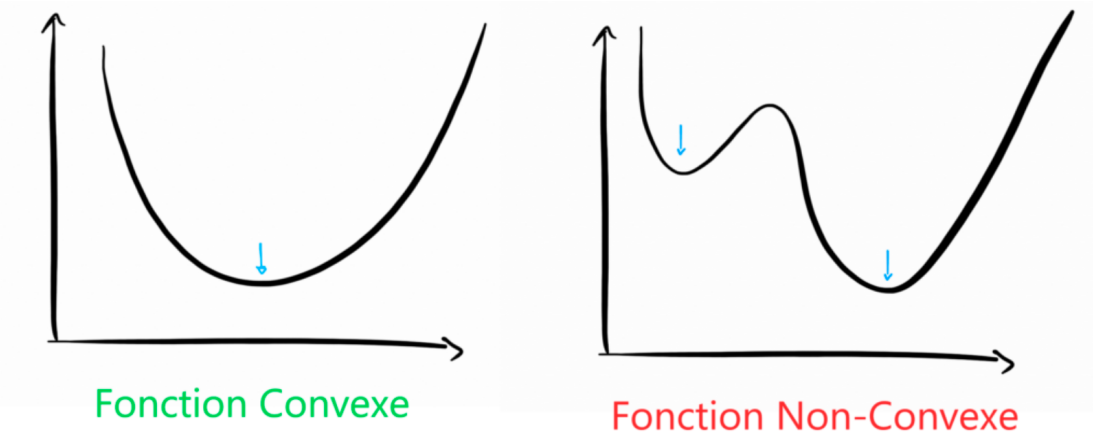
DESCENTE DE GRADIENT

La **Descente de Gradient**, (ou Gradient Descent en anglais) est un des algorithmes les plus importants de tout le Machine Learning et de tout le Deep Learning. il s'agit d'un algorithme d'optimisation extrêmement puissant qui permet d'entraîner les modèles de régression linéaire, régression logistiques ou encore les réseaux de neurones.

DESCENTE DE GRADIENT

La Descente de Gradient est un algorithme d'optimisation qui permet de trouver le minimum de n'importe quelle fonction convexe en convergeant progressivement vers celui-ci.

Exemple :

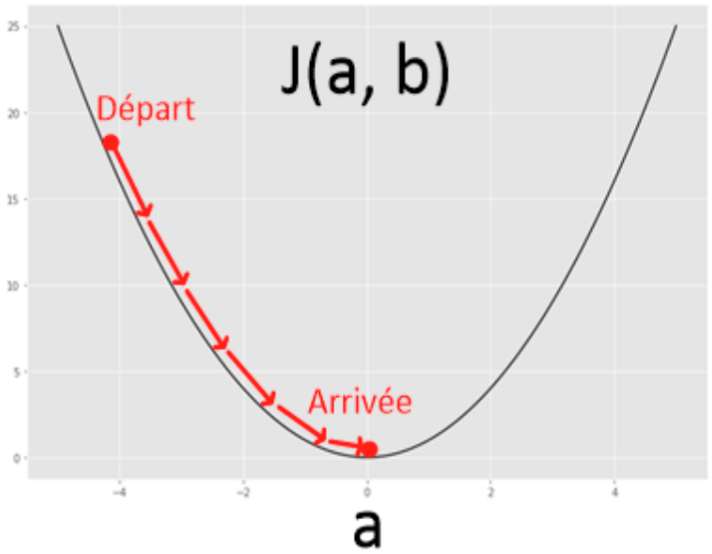


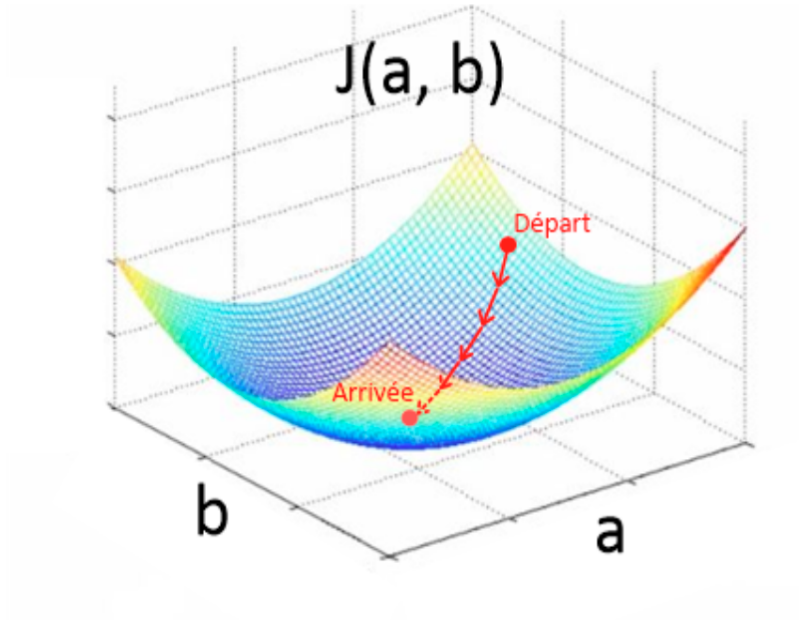
- Une fonction convexe présente un seul minimum global.
- Une fonction non-convexe présente plusieurs minimums locaux.
- L'algorithme de descente de gradient ne doit pas être utilisé sur des fonctions non-convexe, au risque de se bloquer au premier minima

Principe : Imaginez que vous soyez perdu en pleine montagne et que vous voulez rejoindre un refuge situé au point le plus bas de la vallée. En appliquant la stratégie suivante vous aller converger vers le minimum de la vallée.

1. Depuis votre position actuelle, vous cherchez tout autour de vous la direction de là où la pente descend.
2. Une fois que vous avez trouvé cette direction, vous la suivez sur une certaine distance (disons que vous marchez 100 mètres)
3. Répétez l'opération de l'étape 1 et 2.

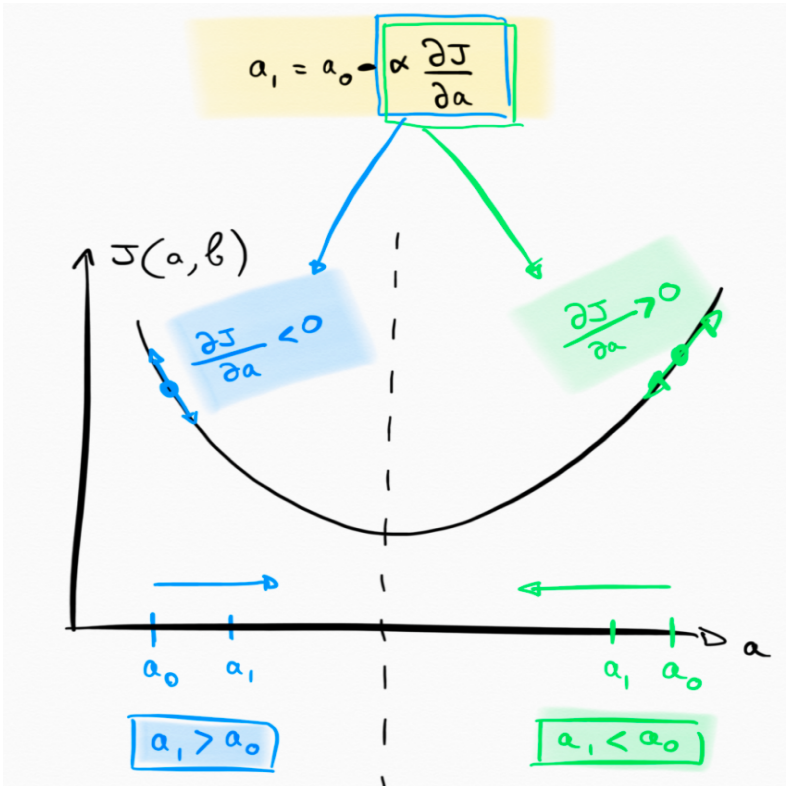
1. Nous partons d'un point initial aléatoire puis nous mesurons la valeur de la pente en ce point (la dérivée).
2. On progresse ensuite d'une certaine distance α (Learning Rate) dans la direction de la pente qui descend.



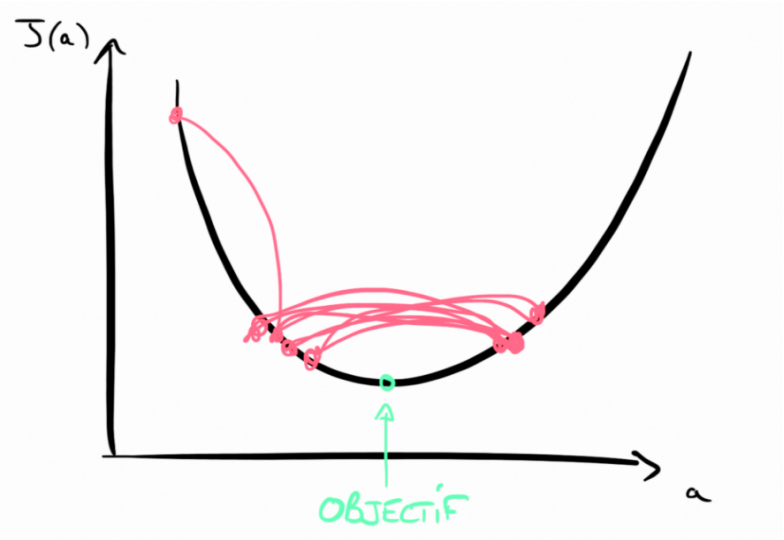


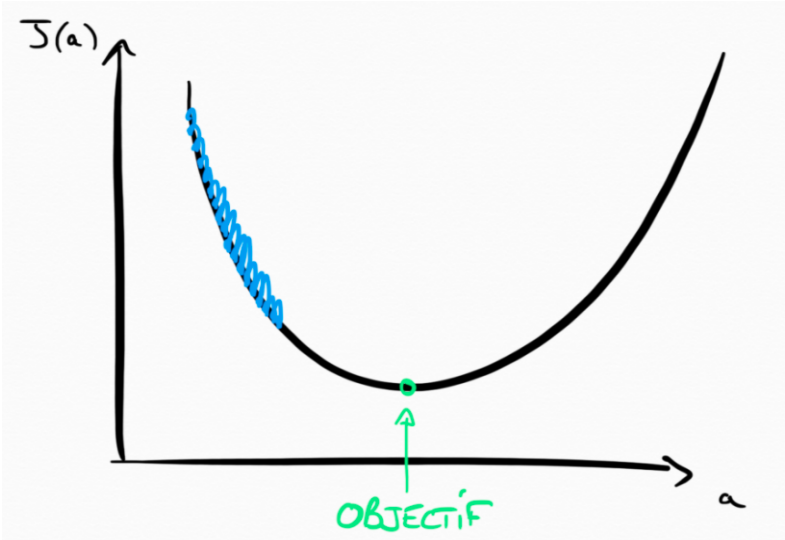
Calculer les nouvelles valeurs de a et b par les formules :

$$a = a - \alpha \frac{\partial J(a,b)}{\partial a} \text{ et } b = b - \alpha \frac{\partial J(a,b)}{\partial b}$$



Choix du coefficient α :





♦ Learning Rate trop grand ⇒ descendre rapidement mais risque d'osciller autour du minimum à l'infini.

♦ Learning Rate trop faible ⇒ temps infini avant de converger.

⇒ Pas de formule magique, on doit tâtonner jusqu'à trouver la bonne valeur

Calcul des dérivés :

On a

$$J(a,b) = \frac{1}{m}\sum_{i=1}^{m}(ax_i + b - y_i)^2.$$

donc

$$\frac{\partial J(a,b)}{\partial a} = \frac{2}{m}\sum_{i=1}^{m} x_i \times (ax_i + b - y_i)$$

$$\frac{\partial J(a,b)}{\partial b} = \frac{2}{m}\sum_{i=1}^{m}(ax_i + b - y_i)$$

$$a = a - \frac{2\alpha}{m} \sum_{i=1}^{m} x_i \times (ax_i + b - y_i)$$

$$b = b - \frac{2\alpha}{m} \sum_{i=1}^{m} (ax_i + b - y_i)$$

L'écriture des équations sous forme matricielle permet :

1. Simplifier les calcules
2. Ne pas avoir besoins de refaire le travail pour des modèles plus complexes, en particulier la régression polynomiale.

Notre modèle peut s'écrire sous la forme matricielle par :

$$F = X\Theta$$

avec

$$F = \begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_m) \end{bmatrix}, X = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \text{ et } \Theta = \begin{bmatrix} a \\ \\ b \end{bmatrix}$$

La fonction coût suivante :

$$J(a,b) = \frac{1}{m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

peut s'écrire sous forme matricielle comme suit :

$$J(\Theta) = \frac{1}{m}\sum(X\Theta - Y)^2$$

avec

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ \end{bmatrix}$$

$$\frac{\partial J(\Theta)}{\partial \Theta} = \frac{2}{m} X^T (X\Theta - Y)$$

Avec

$$\frac{\partial J(\Theta)}{\partial \Theta} = \begin{bmatrix} \frac{\partial J(a,b)}{\partial a} \\ \frac{\partial J(a,b)}{\partial b} \end{bmatrix}$$

et $X^T = \begin{bmatrix} x_1 & x_2 & \cdots & x_m \\ 1 & 1 & \cdots & 1 \end{bmatrix}$ et $X\Theta - Y = \begin{bmatrix} ax_1 + b - y_1 \\ ax_2 + b - y_2 \\ \cdots \\ ax_m + b - y_m \end{bmatrix}$

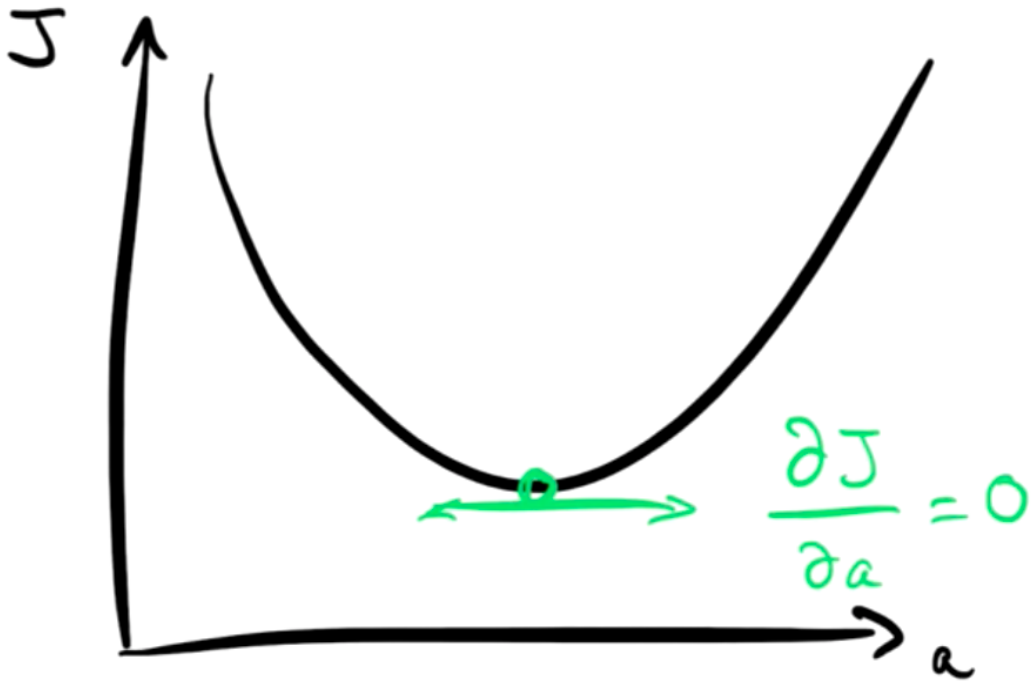
$$\Theta = \Theta - \alpha \frac{\partial J(\Theta)}{\partial \Theta}$$

EXEMPLE EN PYTHON

LA MÉTHODE DES ÉQUATIONS NORMALES

Les équations normales

Les équations normales sont issues de la méthode des moindres carrés. En machine Learning et en statistiques, cette technique permet de résoudre des problèmes de régression en trouvant la solution EXACTE d'un problème de minimisation (c'est la méthode des moindres carrés).



La technique consiste a utiliser l'erreur quadratique moyenne comme fonction cout $J(\Theta)$ à optimiser et de trouver la valeur de Θ où la dérivé de la fonction $J(\Theta)$ s'annule.

$$\frac{\partial J(\Theta)}{\partial \Theta} = \frac{2}{m} X^T (X\Theta - Y)$$

$$\frac{\partial J(\Theta)}{\partial \Theta} = 0$$

$$\Rightarrow \frac{2}{m} X^T (X\Theta - Y) = 0$$

$$\Rightarrow X^T (X\Theta - Y) = 0$$

$$\Rightarrow X^T X\Theta = X^T Y$$

$$\Rightarrow \Theta = (X^T X)^{-1} X^T Y$$

$$\Theta = (X^T X)^{-1} X^T Y$$

Exemple :

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import make_regression
4  x,y = make_regression(n_samples=100,n_features=1, noise=10)
5  y = y.reshape(len(y),1)
6  X = np.hstack((x,np.ones(x.shape)))
7  theta = np.linalg.inv(X.T.dot(X)).dot((X.T.dot(y)))
8  plt.scatter(x,y)
9  predictions = X.dot(theta)
10 plt.plot(x,predictions, c='r')
```

Avantages :

- Simple à programmer
- Permet de trouver la valeur exacte de Θ

Inconvénients :

- Même si c'est rare, il existe des cas où la matrice est inversible.
- Inverser une matrice de grandes dimensions nécessite beaucoup de calcules $\Rightarrow$ on ne peut pas l'appliquer dans des problèmes de machines learnings avec une matrice X de très grandes dimensions.

LA RÉGRESSION LOGISTIQUE

LA RÉGRESSION LOGISTIQUE

Définition :

La régression logistique est une technique prédictive qui vise à construire un modèle permettant de prédire les valeurs prises par une variable cible qualitative (souvent binaire) à partir d'un ensemble de variables explicatives quantitatives.

LA RÉGRESSION LOGISTIQUE

Exemple : Classer un email en tant que '*spam*' ou '*non spam*'.

Dans ce genre de problème, on aura un Dataset contenant une variable target y pouvant prendre 2 valeurs, par exemple 0 ou 1

- si $y = 0$, alors l'email n'est pas un spam
- si $y = 1$, alors l'email est un spam

On dit également que l'on a 2 classes, c'est une classification binaire.

LA RÉGRESSION LOGISTIQUE

Considérons une entrée

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix}$$

La régression logistique a pour objectif de trouver une fonction f telle que :

$$f(x) = y$$

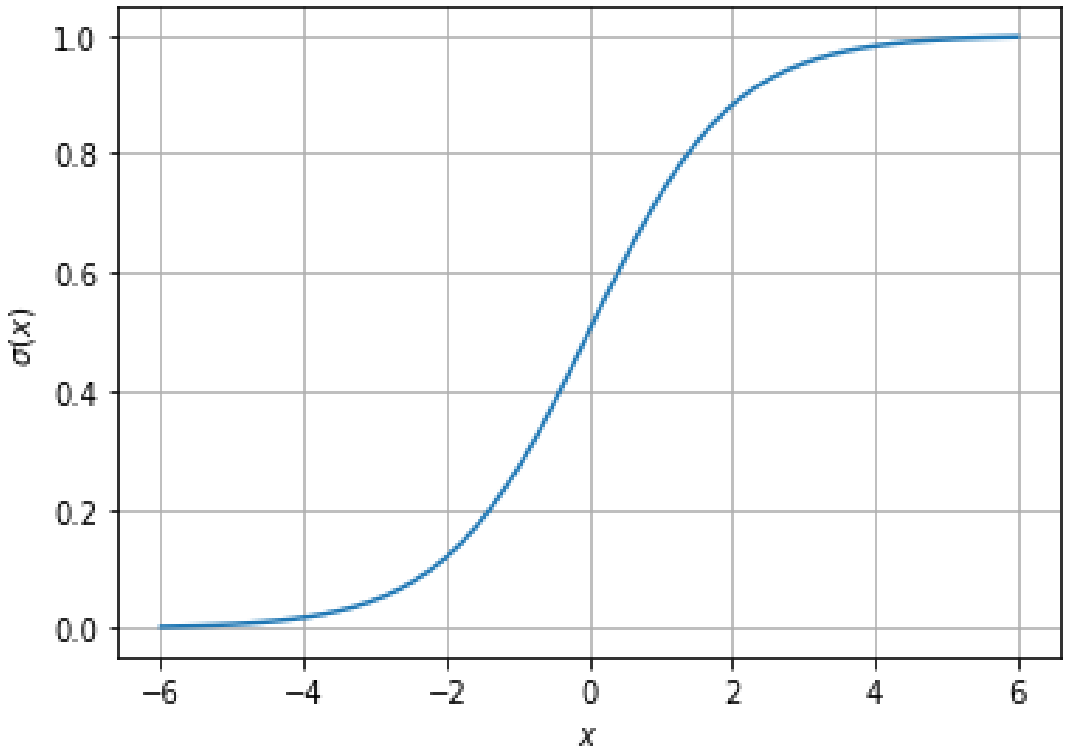
où

$$y = \begin{cases} 1 & \text{si } f(x) \geq \textit{seuil} \\ 0 & \text{si } f(x) < \textit{seuil} \end{cases}$$

- On comprend donc qu'on attend de la fonction f qu'elle soit une probabilité comprise entre 0 et 1,
- Le seuil que nous définissons correspond au critère de classification (généralement 0.5)

La fonction qui remplit le mieux ces conditions est la fonction **logistique** (aussi appelé fonction **sigmoïde** ou tout simplement sigma σ), définie sur $\mathbb{R}$ à valeurs dans $[0, 1]$.

$$\sigma(x) = \frac{1}{1 + np.exp(-x)}$$



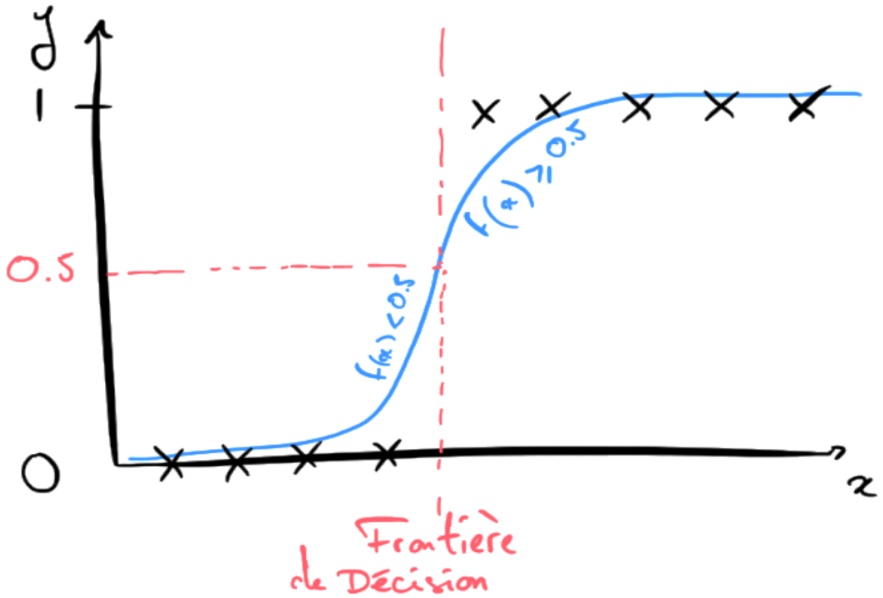
Pour coller la fonction logistique sur un Dataset (X, y) on y fait passer le produit matriciel $X.\Theta$ ce qui nous donne le modèle de **Logistic Regression** :

$$\sigma(X.\Theta) = \frac{1}{1 + e^{-X.\Theta}}$$

A partir de cette fonction, il est possible de définir une frontière de décision.

Typiquement, on définit un seuil à 0.5 comme ceci :

$$y = \begin{cases} 1 & \text{si } \sigma(X.\Theta) \geq 0.5 \\ 0 & \text{si } \sigma(X.\Theta) < 0.5 \end{cases}$$



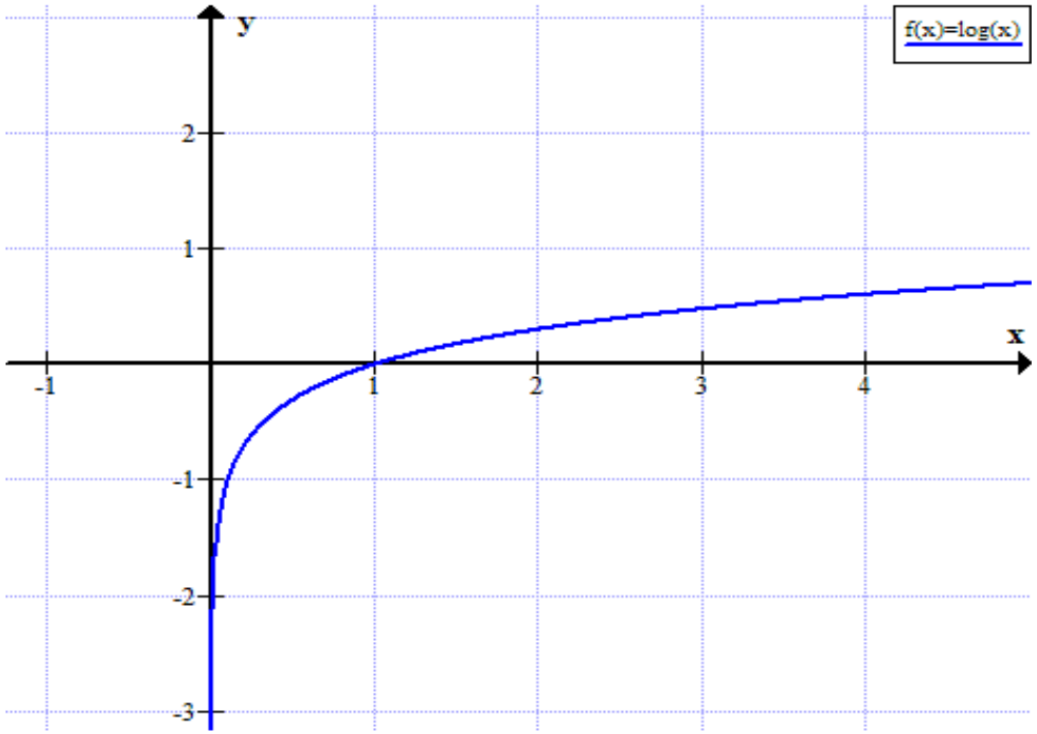
Pour la régression linéaire, la Fonction Coût

$$J(\Theta) = \frac{1}{m}\sum(X\Theta - Y)^2$$

donnait une courbe convexe (qui présente un unique minima). C'est ce qui fait que l'algorithme de Gradient Descent fonctionne.
En revanche, utiliser cette fonction pour le modèle Logistique ne donnera pas de courbe convexe et l'algorithme de Gradient Descent se bloquera au premier minima rencontré, sans trouver le minimum global.

Solution :

Utilise la fonction logarithme pour transformer la fonction σ en fonction convexe en séparant les cas où $y = 1$ des cas où $y = 0$.



Fonction Coût dans les cas où $y = 1$

$$J(\Theta) = -log(\sigma(X.\Theta))$$

Explication :

Si notre modèle prédit $\sigma(x) = 0$ alors que $y = 1$, on doit pénaliser la machine par une grande erreur (un grand coût). La fonction logarithme permet de tracer cette courbe avec une propriété convexe, ce qui poussera le Gradient Descent à trouver les paramètres Θ pour un coût qui tend vers 0.

Fonction Coût dans les cas où $y = 0$

$$J(\Theta) = -log(1 - \sigma(X.\Theta))$$

Explication :

Si notre modèle prédit $\sigma(x) = 1$ alors que $y = 0$, on doit pénaliser la machine par une grande erreur (un grand coût).

Pour écrire la Fonction Coût en une seule équation, on utilise l'astuce de séparer les cas $y = 0$ et $y = 1$ avec une annulation :

$$J(\Theta) = \frac{-1}{m} \sum y \times log(\sigma(X.\Theta)) + (1 - y) \times log(1 - \sigma(X.\Theta))$$

L'algorithme de Gradient Descent s'applique exactement de la même manière que pour la régression linéaire. En plus, la dérivée de la Fonction Coût est la même aussi :

$$\frac{\partial J(\Theta)}{\partial \Theta} = \frac{1}{m} X^T (\sigma(X.\Theta) - Y)$$

$$\Theta = \Theta - \alpha \frac{\partial J(\Theta)}{\partial \Theta}$$

EXEMPLE EN PYTHON

LA CLASSIFICATION : K-Nearst Neighbor (KNN)

K Nearest Neighbors (KNN)

C'est un algorithme d'apprentissage supervisé. Il peut être utilisé aussi bien pour la **régression** que pour la **classification**. Son fonctionnement peut être assimilé à l'analogie suivante "*dis moi qui sont tes voisins, je te dirais qui tu es*".

Pour effectuer une prédiction, l'algorithme KNN ne va pas calculer un modèle prédictif à partir d'un Training Set comme c'est le cas pour la régression logistique ou la régression linéaire.

- Pas besoin de construire un modèle prédictif.
- Pas de phase d'apprentissage proprement dite.
- Il ne nécessite que le stockage de jeu de données d'apprentissage.

KNN : K Nearst Neighbor

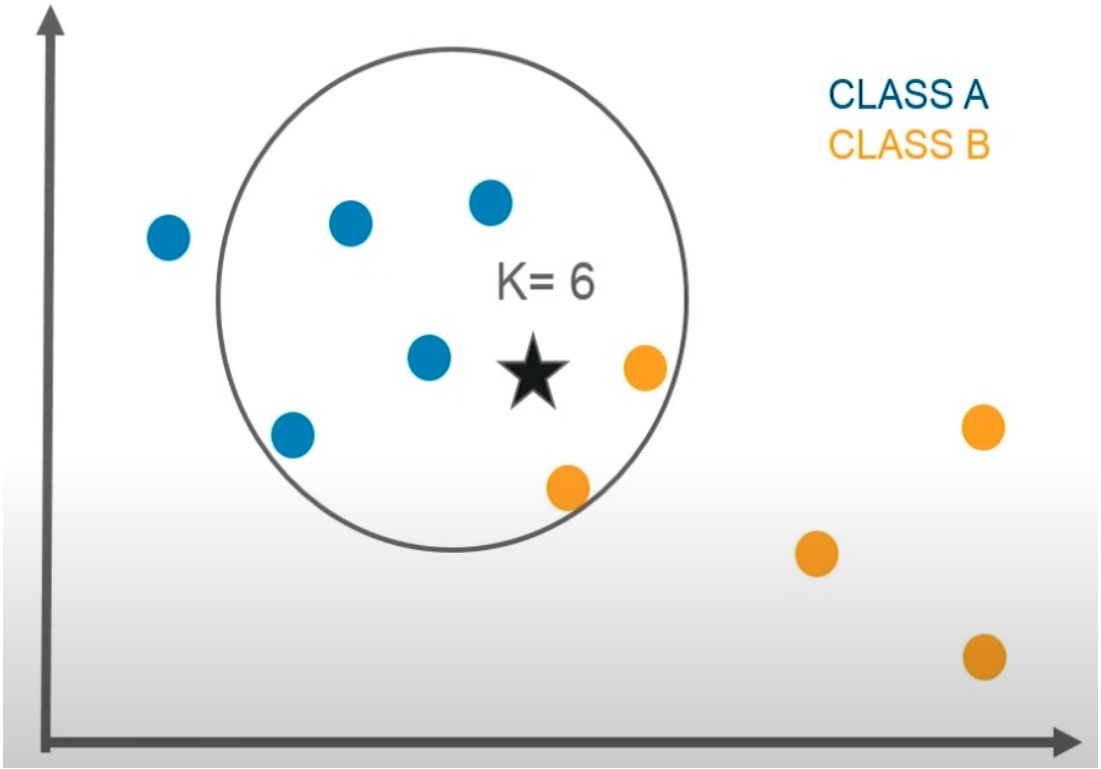
Principe de l'algorithme

Cet algorithme est qualifiée comme paresseux (Lazy Learning) car il n'apprend rien pendant la phase d'entraînement.

En effet, pour prédire la classe d'une nouvelle donnée d'entrée, il cherche ses k voisins les plus proches (en utilisant une fonction distance) et choisi la classe des voisins majoritaires.

KNN : K Nearst Neighbor

Principe de l'algorithme



KNN : K Nearst Neighbor

Principe de l'algorithme

Données en entrée :

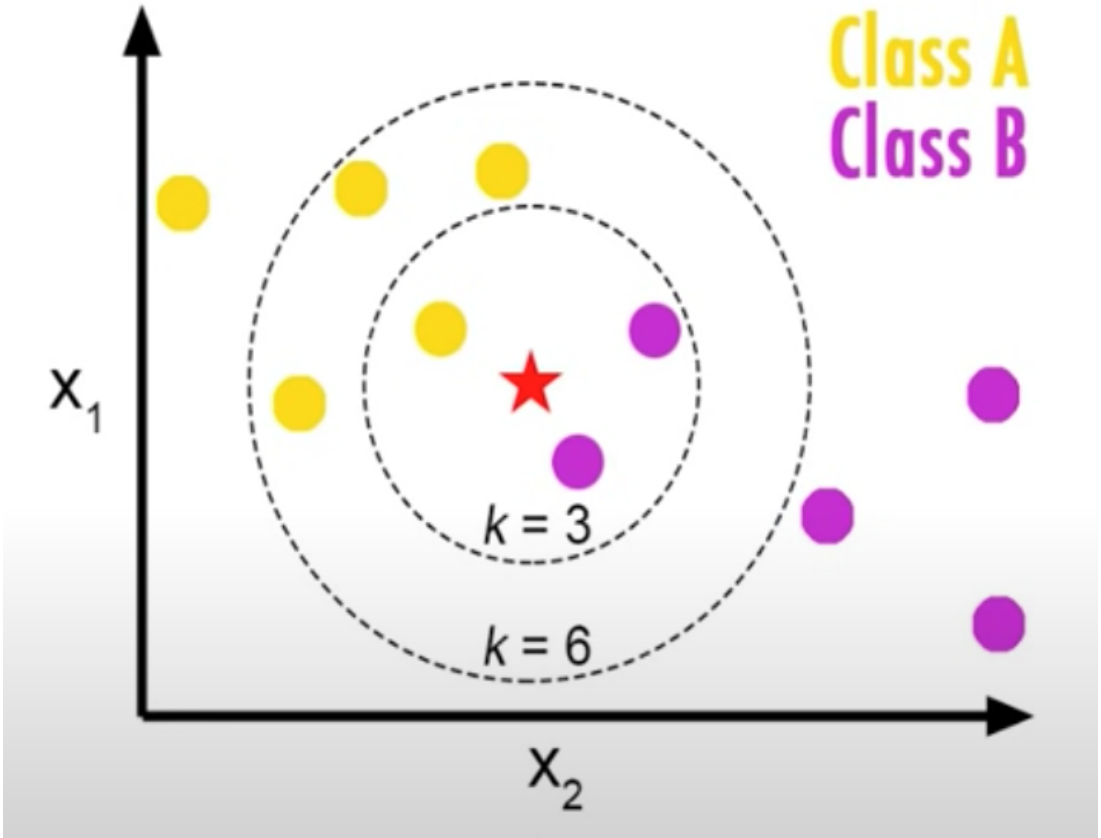
- Un ensemble de données D .
- Une fonction de définition distance d.
- Un nombre entier K.

Pour une nouvelle observation x dont on veut prédire sa variable de sortie y :

1. Calculer toutes les distances de cette observation x avec les autres observations du jeu de données D
2. Retenir les k données du jeu de données E les plus proches de x.
3. Attribuer à x la classe qui est la plus fréquente parmi les k données les plus proches.

KNN : K Nearst Neighbor

Choix de K



KNN : K Nearst Neighbor

Distances

Soit $x = \{x_1, x_2, ..., x_n\}$ et $y = \{y_1, y_2, ..., y_n\}$ deux observations :

1. **Distance Euclidienne :**

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

2. **Distance de Manhattan :**

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

3. **Distance de Minkowski :**

$$d(x, y) = \sqrt[q]{\sum_{i=1}^{n} |x_i - y_i|^q}$$

KNN : K Nearst Neighbor

Propriétés d'une distances

1. $d(x, y) \geq 0$
2. $d(x, x) = 0$
3. $d(x, y) = d(y, x)$
4. $d(x, z) \leq d(x, y) + d(y, z)$

KNN : K Nearst Neighbor

Exemples de domaines d'application

- Il est utilisé pour prédire si la banque doit accorder un prêt à un particulier. Il tentera d'évaluer si l'individu donné correspond aux critères des personnes qui avaient précédemment fait défaut ou ne fera pas défaut à son prêt.
- Elaboration d'un profil pour proposer des films appropriés (Netflix, Youtube, ...).
- Classer un électeur potentiel dans les catégories «votera» ou «ne votera pas» pour tel ou tel candidat.

KNN : K Nearst Neighbor

Avantages :

1. Simple et facile à mettre en œuvre.
2. Il n'est pas nécessaire de créer un modèle ou de régler plusieurs paramètres.
3. Il est polyvalent et peut être utilisé pour la *classification* ou la *régression*.

Inconvénients :

1. Le coût en termes de temps de réponse et de l'espace mémoire peut devenir important. En effet, il doit garder en mémoire l'ensemble des observations pour pouvoir effectuer sa prédiction.
2. Le choix de la méthode de calcul de la distance ainsi que le nombre de voisins k peut ne pas être évident.

APPRENTISSAGE NON-SUPERVISÉ

Apprentissage Non-Supervisé vs Apprentissage Supervisé

Apprentissage Supervisé :

On montre à la machine des exemples (X, y) de ce quelle doit apprendre.

Apprentissage Non-Supervisé vs Apprentissage Supervisé

Apprentissage Non-Supervisé :

La machine analyse la structure des données X pour apprendre toute seule à réaliser certaines taches.

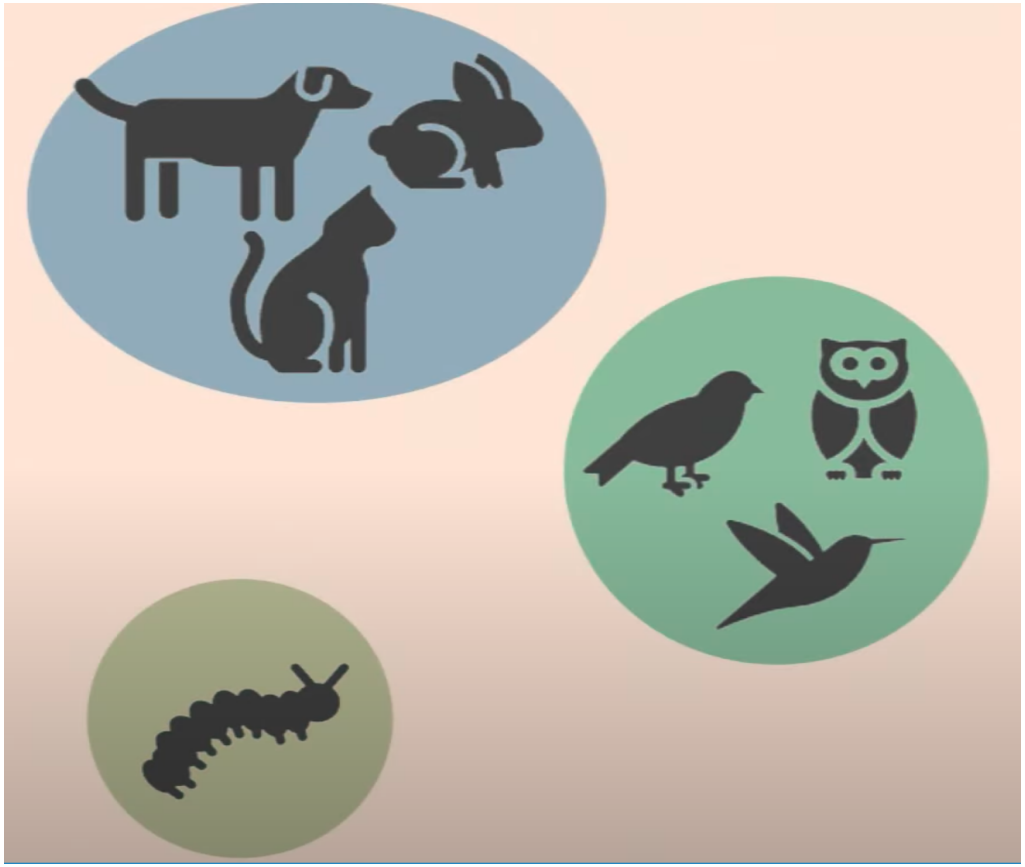
Apprentissage Non-Supervisé

Applications de l'apprentissage Non-Supervisé :

1. Clustering
2. Détection d'anomalies
3. Réduction de dimensions

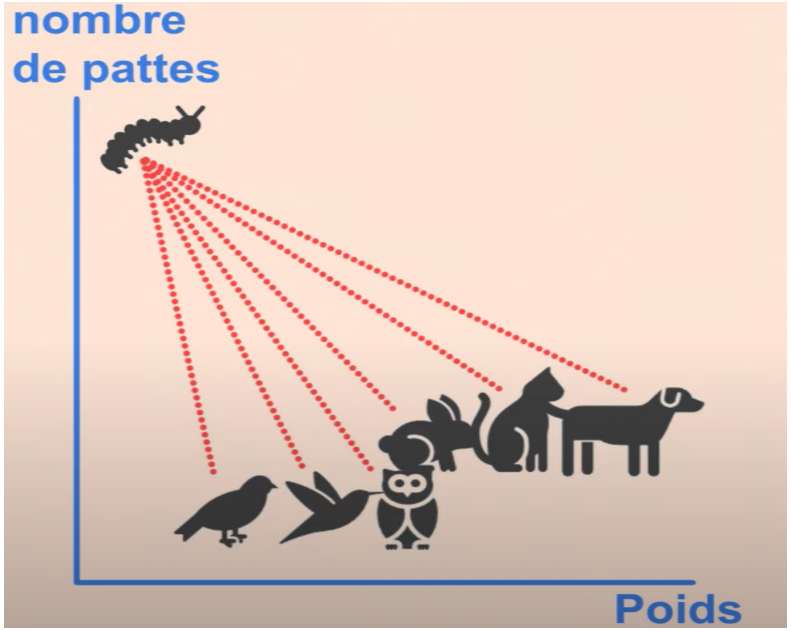
Clustering :

Regrouper les données en se basant sur leurs ressemblances :



Apprentissage Non-Supervisé

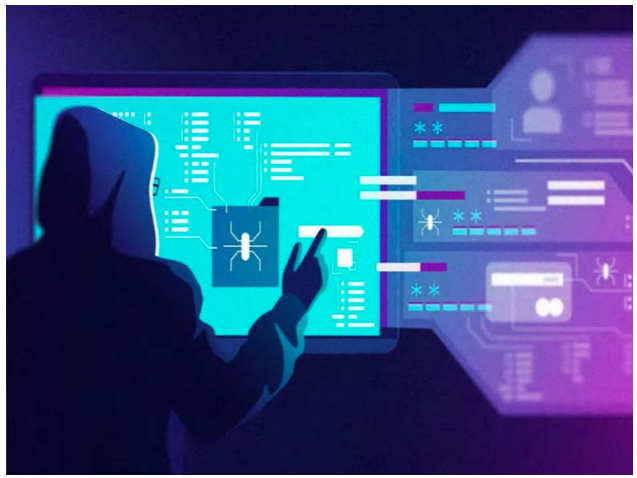
Détection d'anomalies : Trouver les échantillons dont les caractéristiques (X) sont très éloignées de celles des autres échantillons.



$\Rightarrow$ Algorithme : **Isolation Forest**.

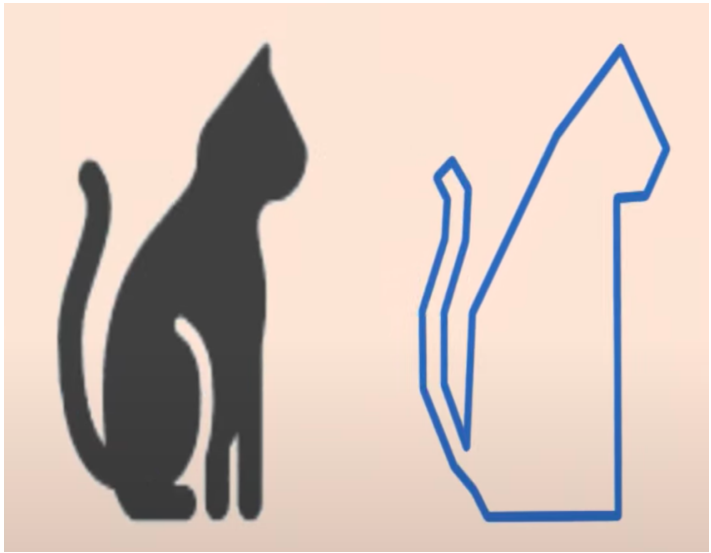
Détection d'anomalies : Applications

- Système de sécurité.
- Défaillance dans une usine.
- Détection de fraudes bancaires.
- etc



Apprentissage Non-Supervisé

Réduction de dimensions : En analysant la structure de nos données, la machine apprend à la simplifier tout en conservant les principales informations.



Pour réaliser cela, la machine apprend à projeter le mieux possible nos données dans des espaces de plus petite dimensions.

$\Rightarrow$ Algorithme : **Analyse en Composante Principale (PCA)**.

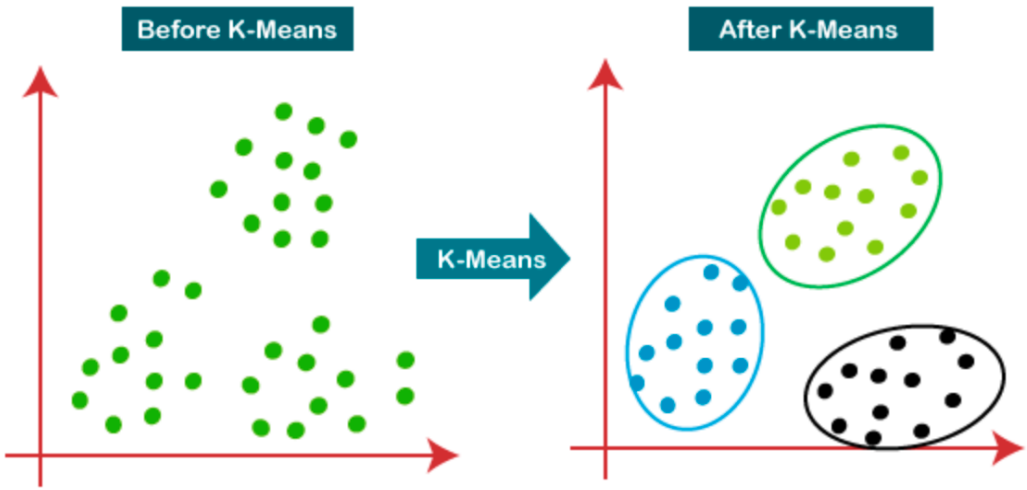
Le clustering

K-Means

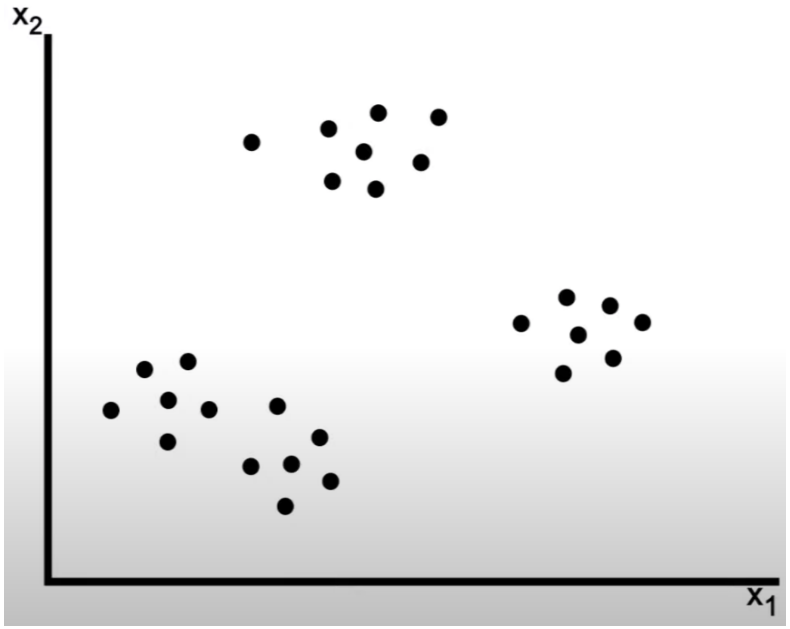
Définition

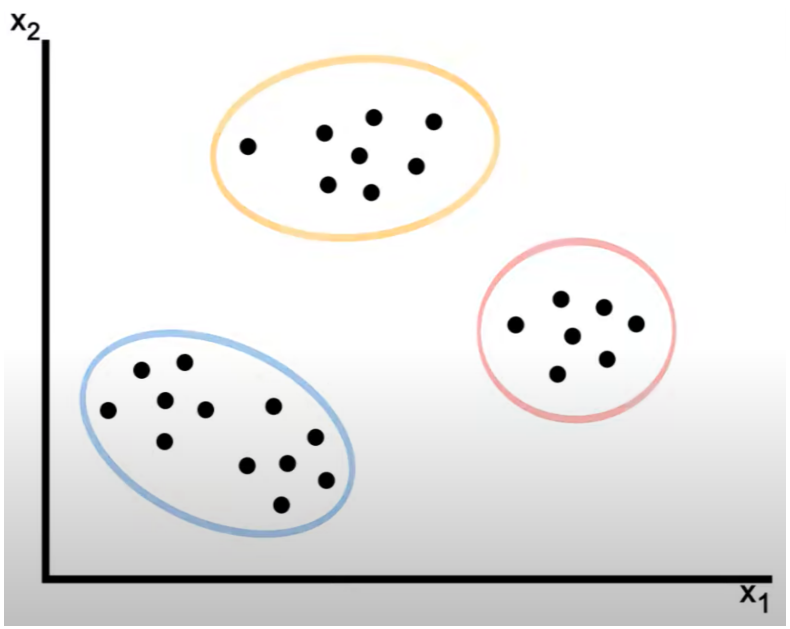
K-means is one of the most popular Unsupervised Machine Learning Algorithms Used for Solving clustering Problems.

K-Means segregates the unlabeled data into various groups, called clusters, based on having similar features, common patterns.

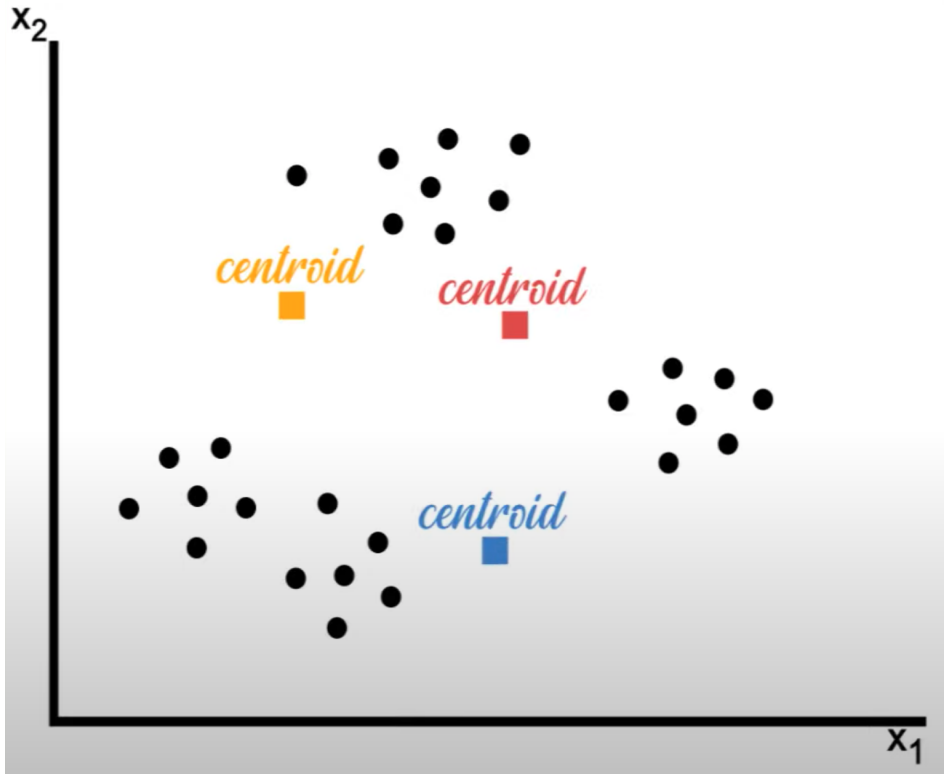


Principe : Imaginons que nous voulons regrouper les données suivantes en trois clusters.

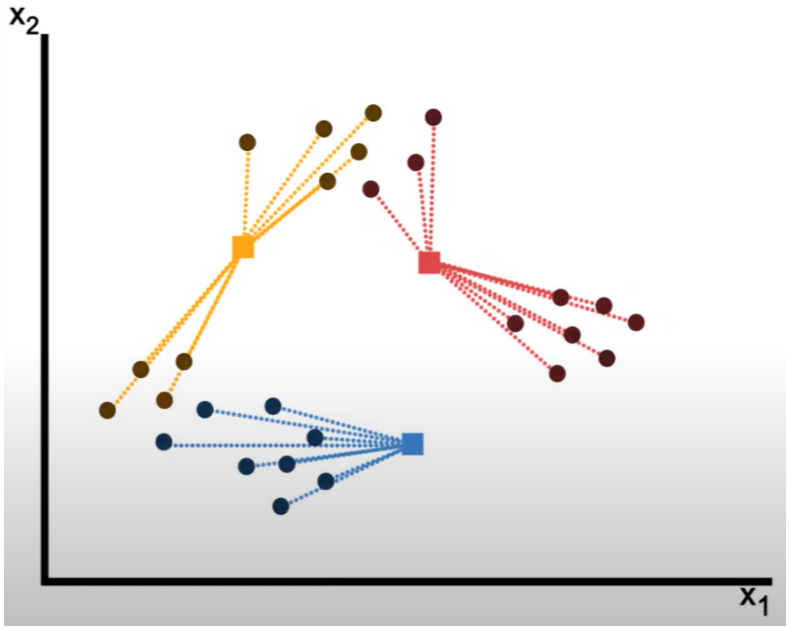


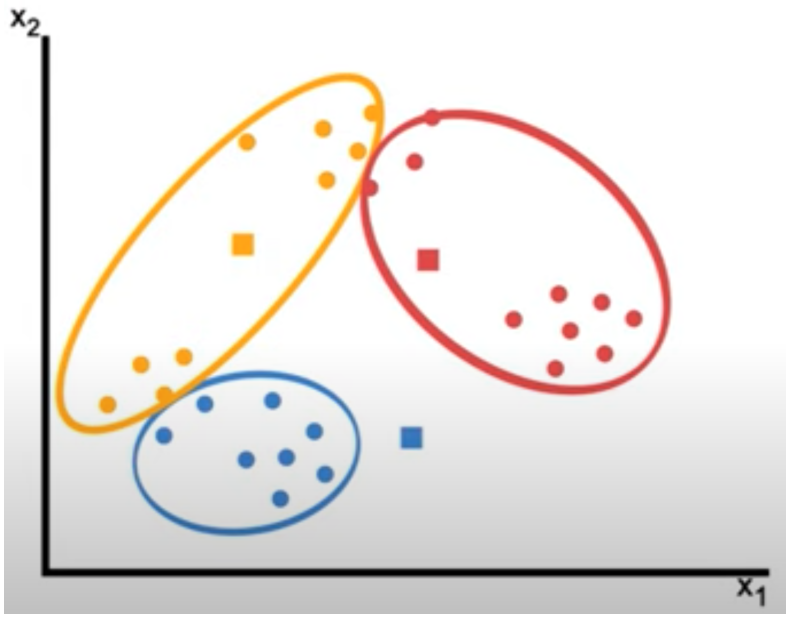


◇ **Étape 1 :** On place K (ici K=3) centroides au hasard dans le dataset.

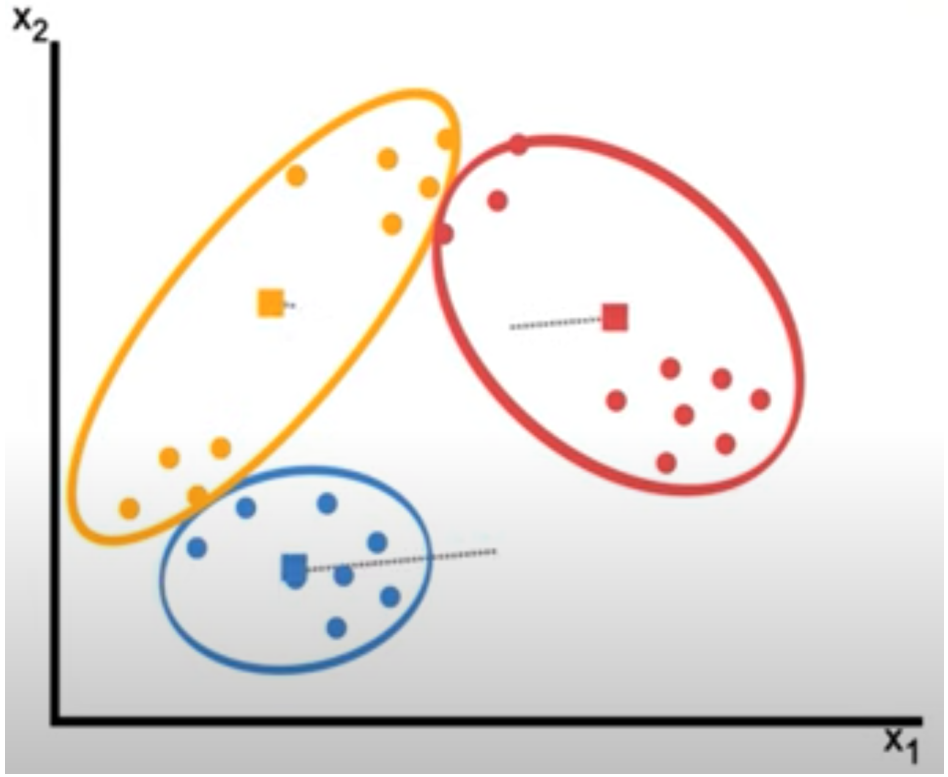


◇ **Étape 2 :** On calcule la distance de chaque point de dataset aux différents centroids et on l'affecte au centroid le plus proche.

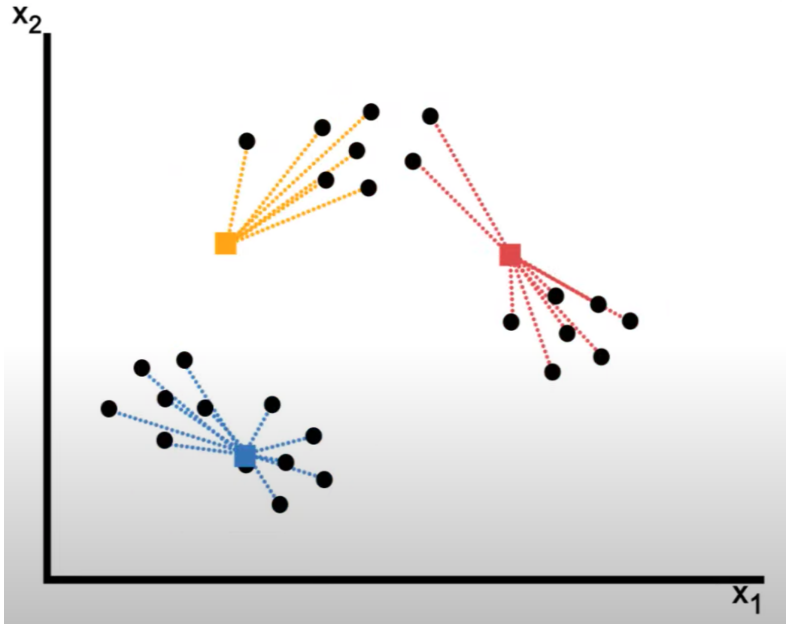


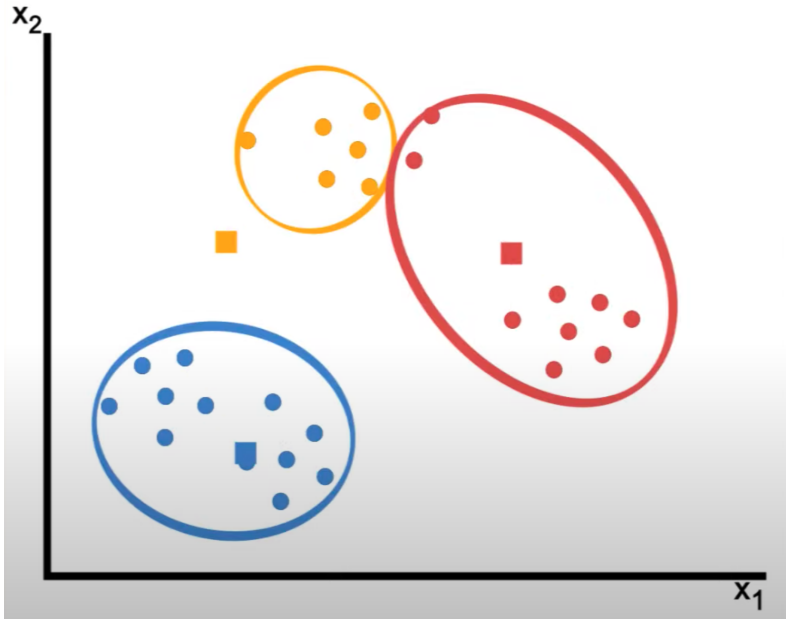


◇ **Étape 3 :** On déplacer (on recalcule) chaque centroid au milieu de son cluster (la où se situe la moyenne des points).



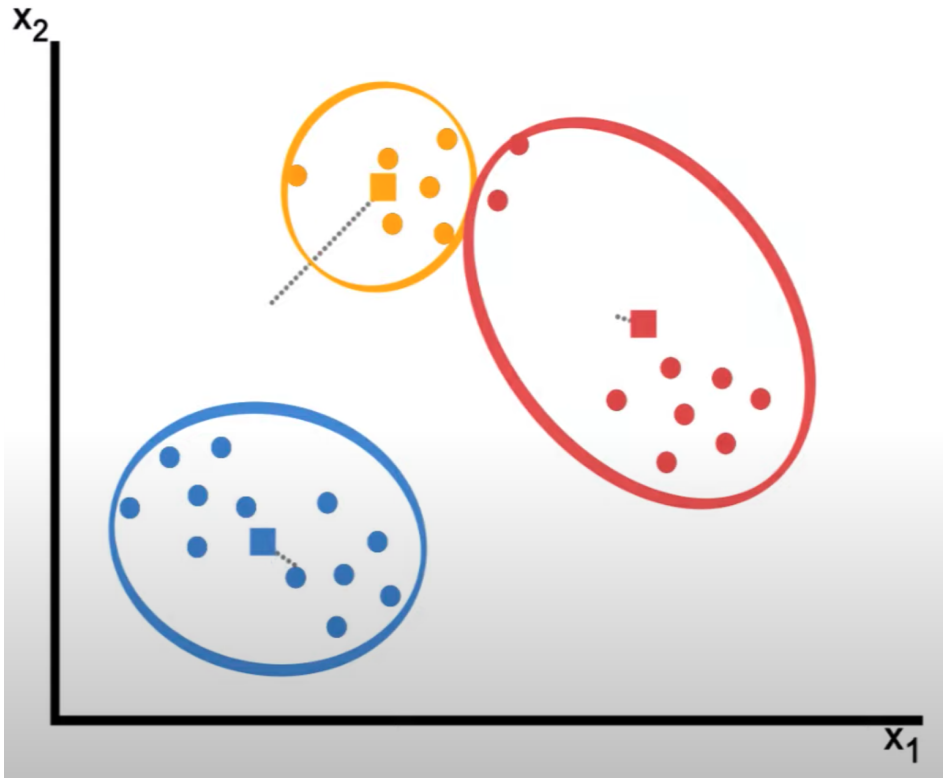
◇ **Étape 4 :** On recommence l'étape 2. *On calcule la distance de chaque point de dataset aux différents centroids et on l'affecte au centroid le plus proche.*



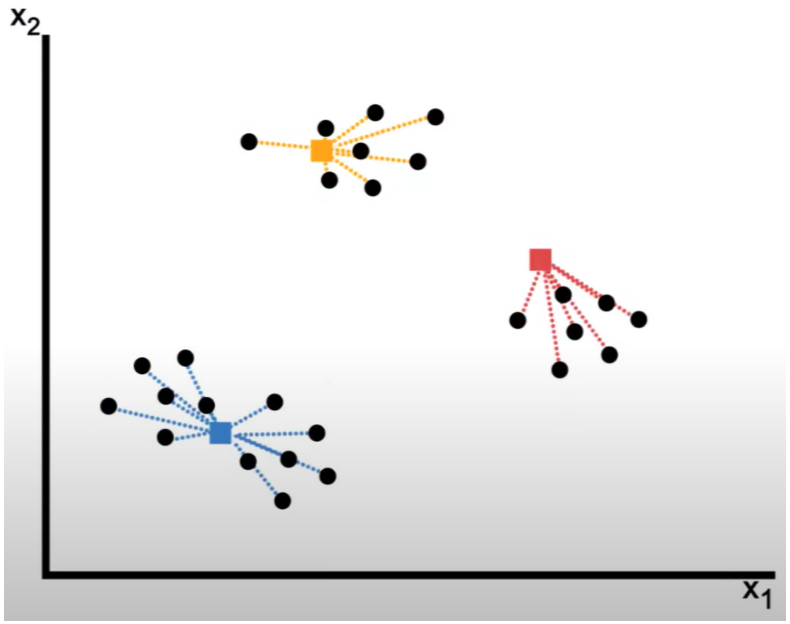


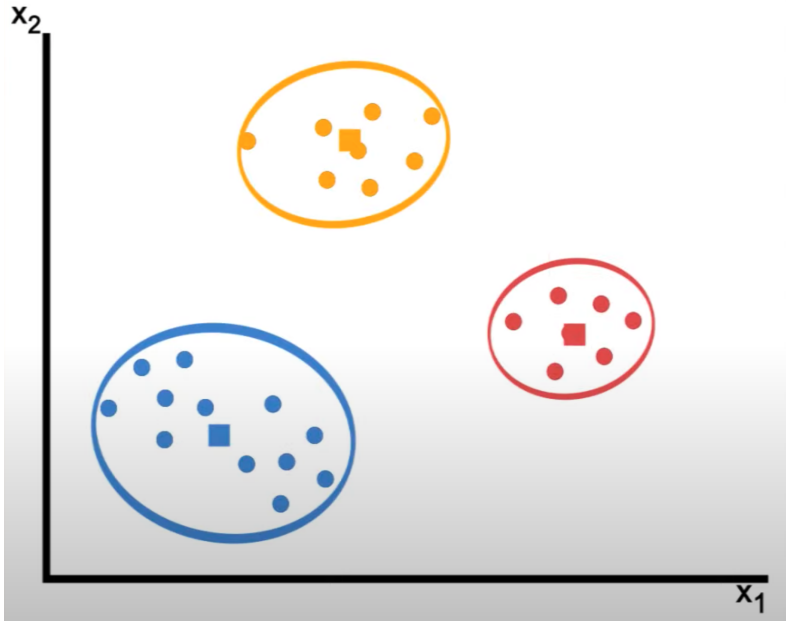
◇ **Étape 5 :** On recommence l'étape 3.

On déplacer (on recalcule) chaque centroid au milieu de son cluster (la où se situe la moyenne des points).



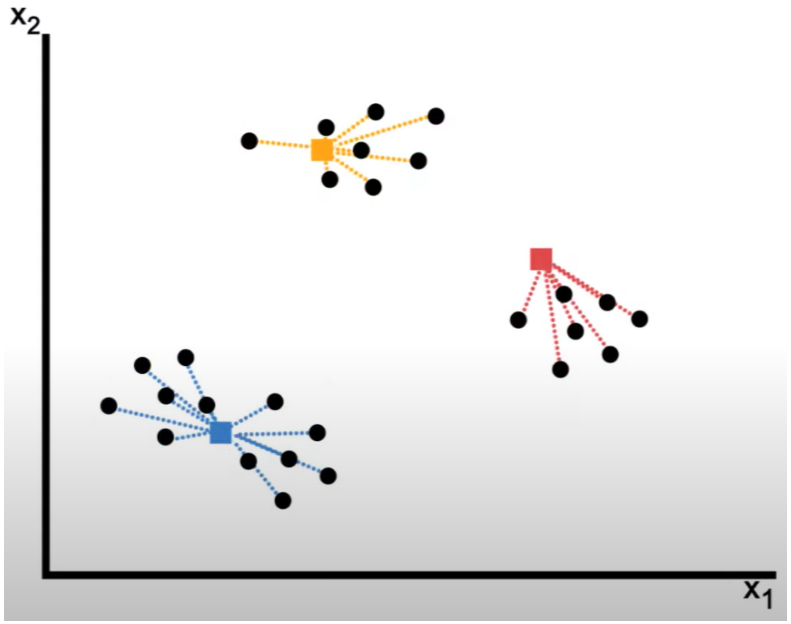
◇ **Étape 6 :** On continue ainsi jusqu'à ce que les centroids convergent vers une position d'équilibre



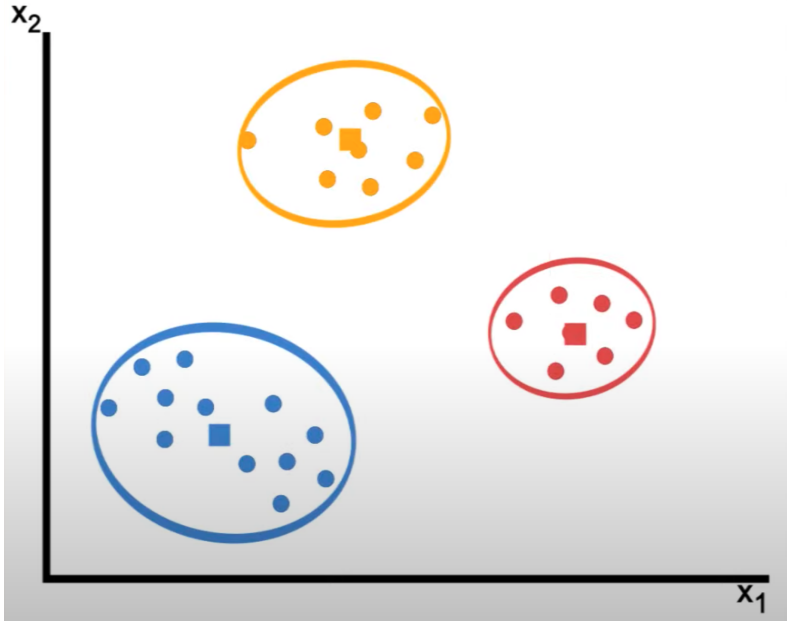


En gros : K-Means est un algorithme itératif qui fonctionne en deux étapes répétitifs :

1. Affectation des points au centre le plus proche

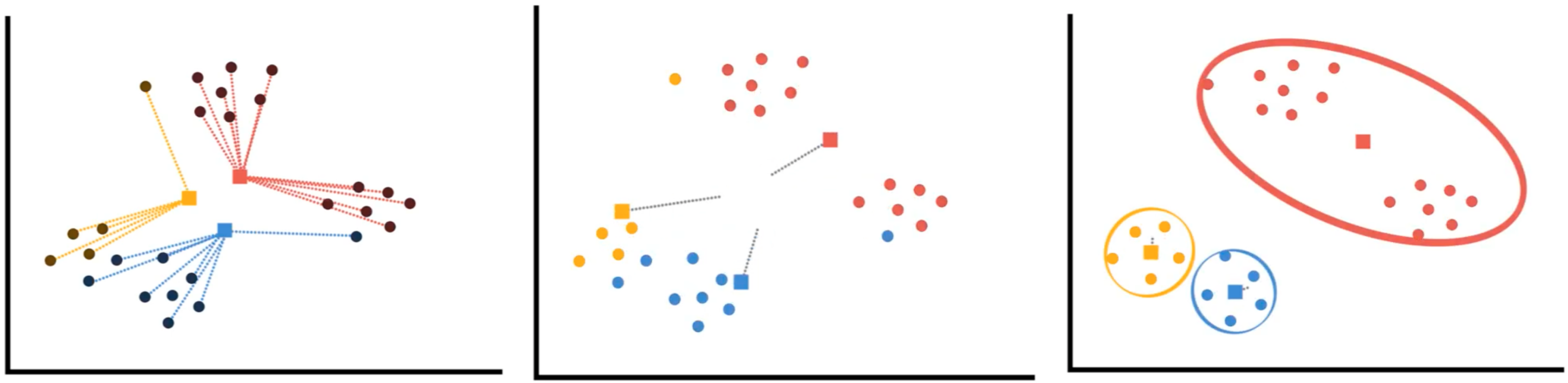


2. Déplacer les centroids aux moyennes des clusters.



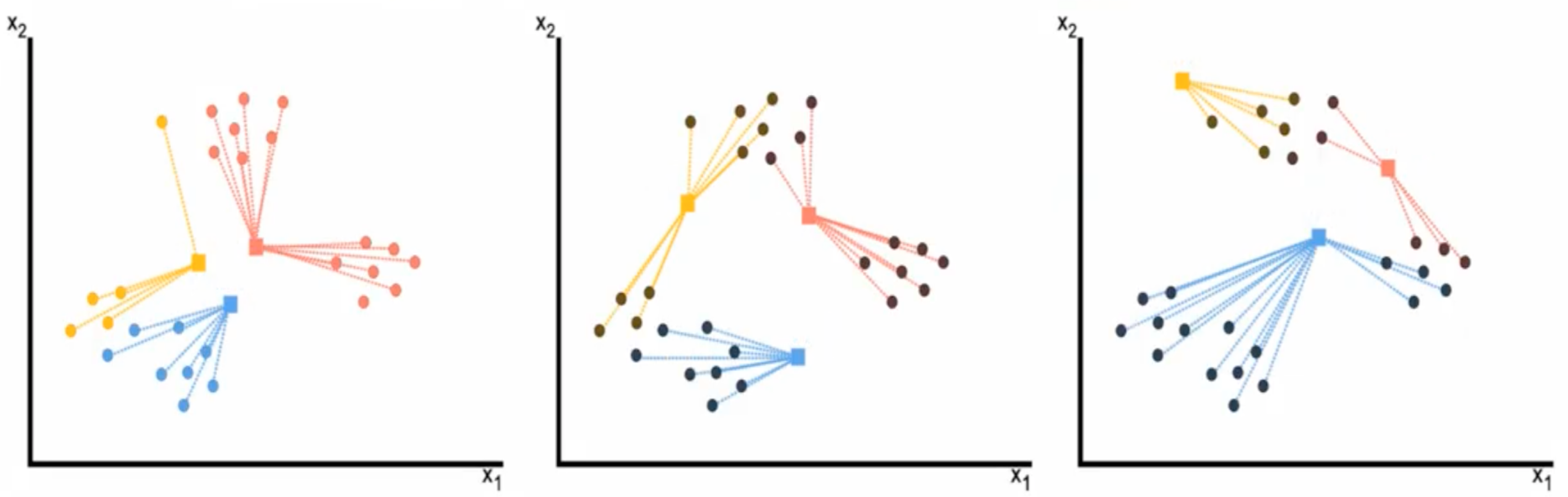
Mauvais centroids :

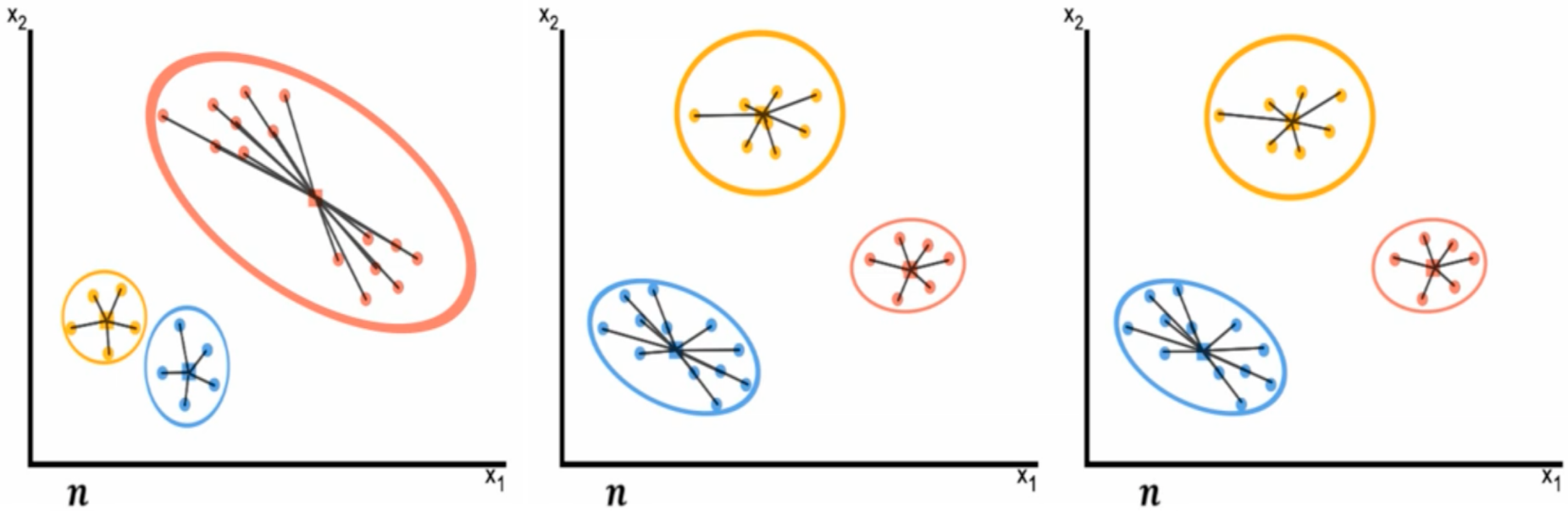
Selon les postions initiales des centroids il est possible qu'il converge vers de mauvaises positions.



Solution :

On execute K-Means avec différentes positions initiales, et on retient la solution qui minimise la somme des distances entre les différents points d'un cluster et son centre.

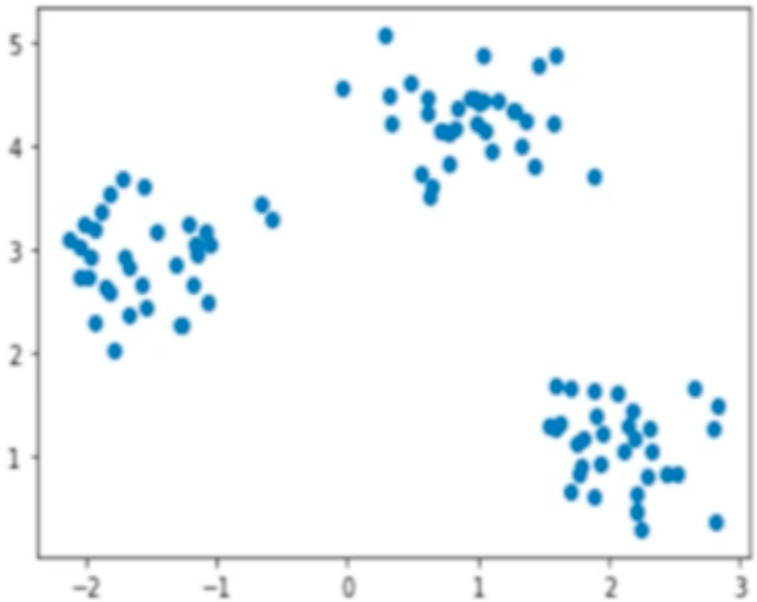




La fonction coût *Inirtia*

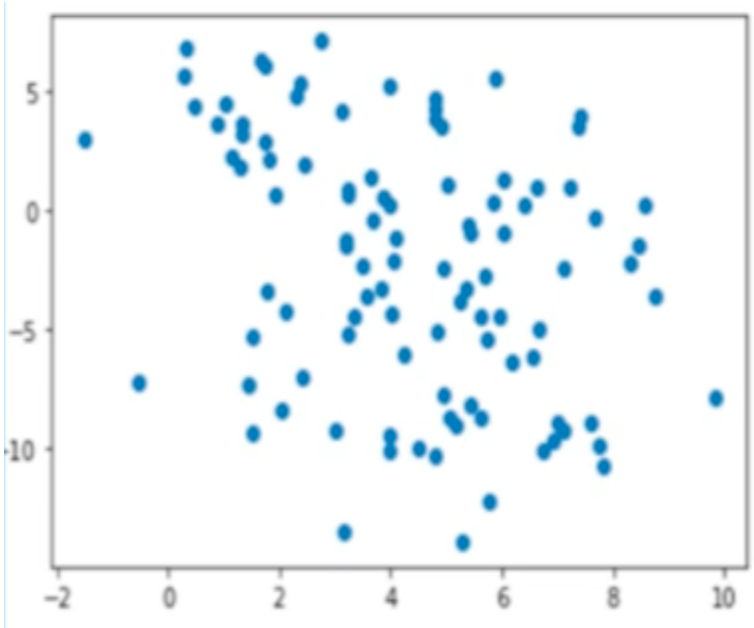
$$\sum_{j=1}^{k} \sum_{i=1}^{n} |x_i - \mu_j|^2$$

Comment choisir la bonne valeur de K ?



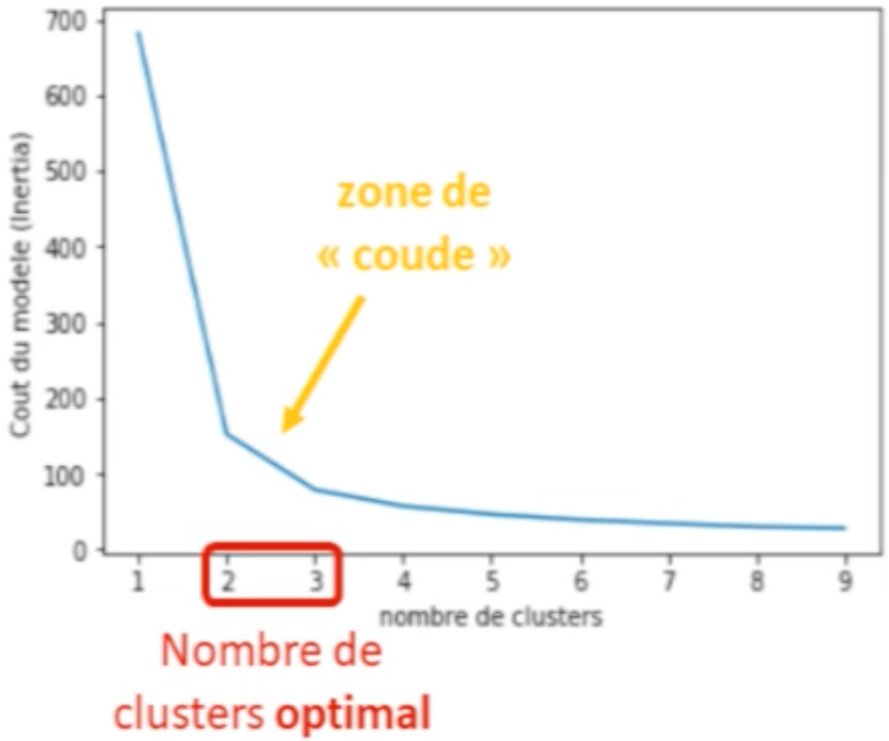
Facile à définir le nombre de clusters (ici k=3)

Comment choisir la bonne valeur de K ?



1. Le dataset est à plusieurs dimensions
2. Difficile de voir les clusters à l'œil.
3. etc

Solution : La méthode Elbow Pour trouver le bon nombre de clusters, la méthode elbow détecte une zone de **coude** dans la minimisation de coût *inirtia*.

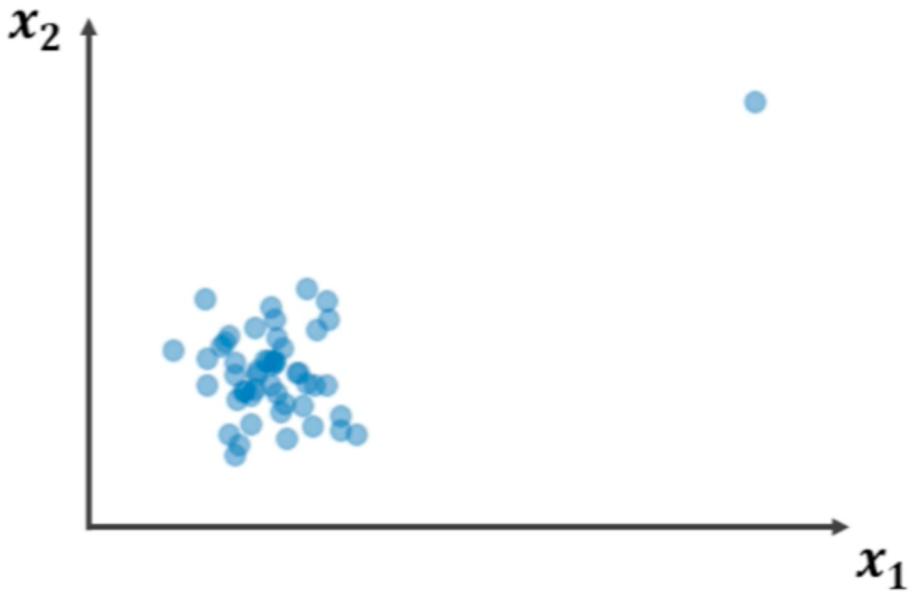


Détection d'anomalies

Isolation Forest

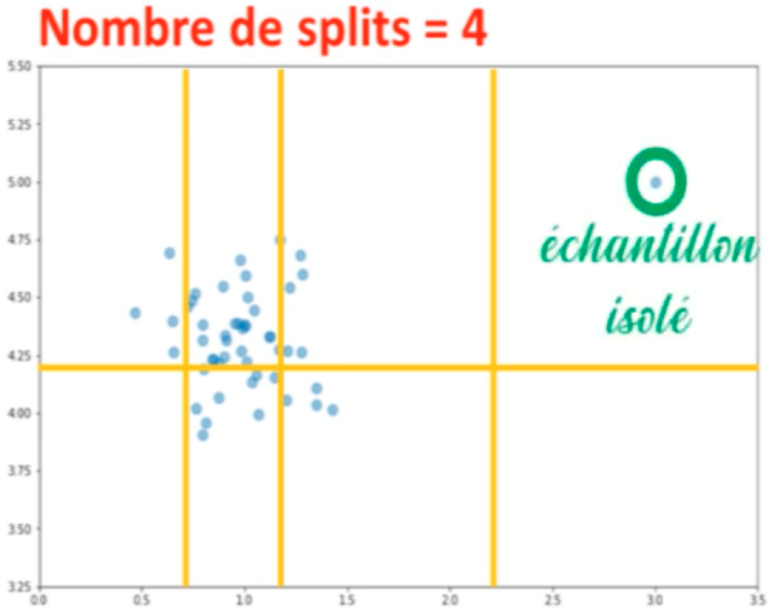
Détection d'anomalies

La détection d'anomalie est une technique d'apprentissage non supervisée qui consiste à détecter dans notre dataset les échantillons dont les caractéristiques sont très éloignés de celles des autres échantillons.



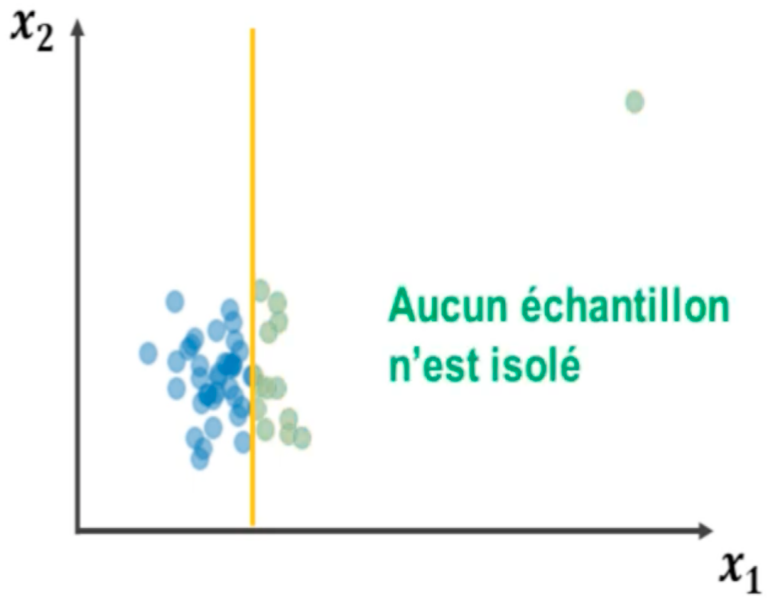
Solution :

On effectue une série de splits aléatoires et on compte le nombre de splits qu'il faut effectuer pour pouvoir isoler nos échantillons.



Faible nombre de splits ⇒ Anomalie

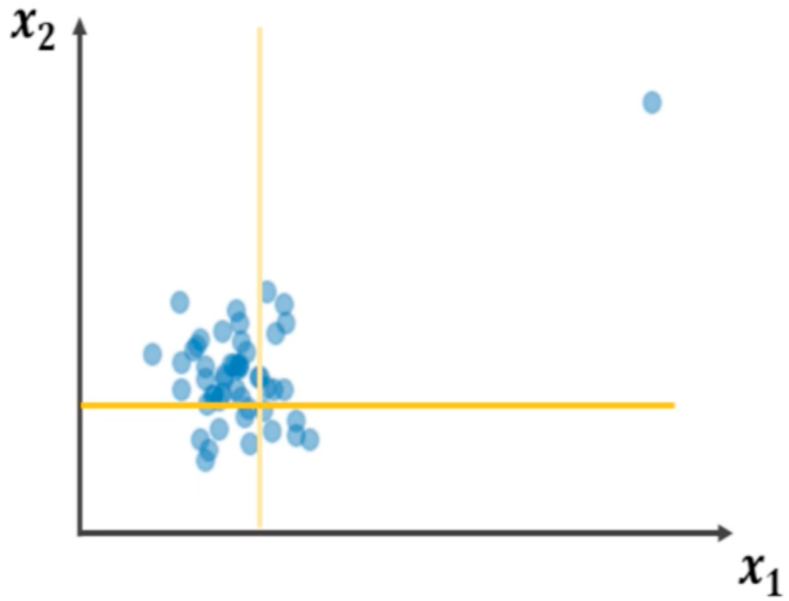
Principe : On choisis au hasard une des variables (par exemple x_1)et on trace quelque part (au hasard) une trait orthogonale à cet axe. On vérifie si ce trait nous a permis d'isoler un de nous échantillons.



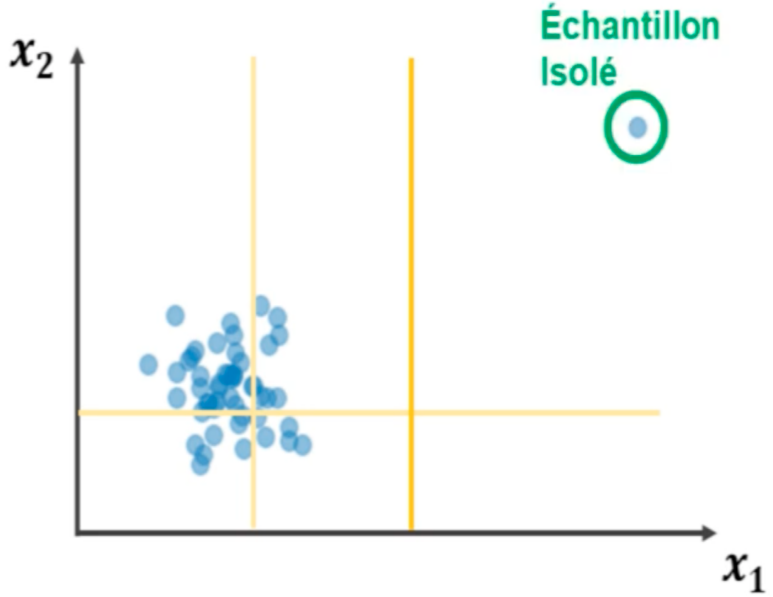
Détection d'anomalies

Isolation Forest

Principe : On choisis au hasard une des variables (par exemple x_2)et on trace quelque part (au hasard) une trait orthogonale à cet axe. On vérifie si ce trait nous a permis d'isoler un de nous échantillons.

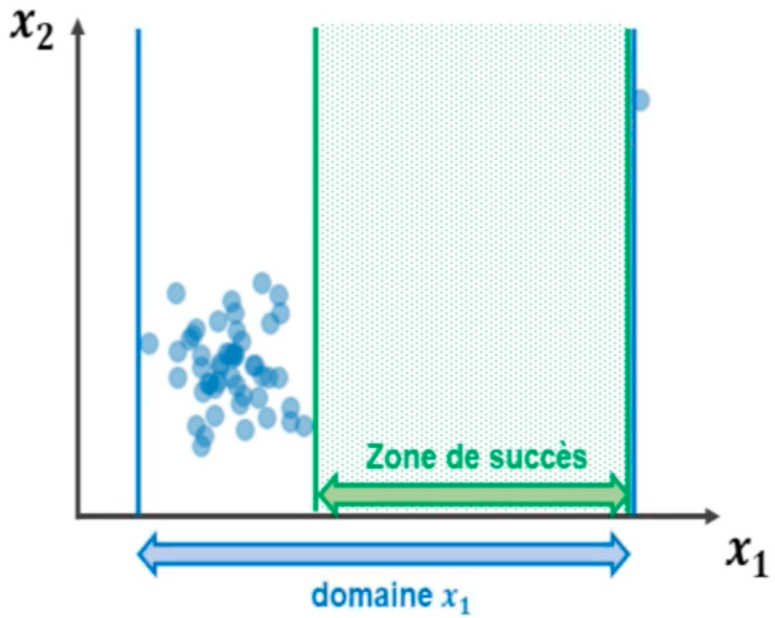


Principe : On choisis au hasard une des variables (par exemple x_1)et on trace quelque part (au hasard) une trait orthogonale à cet axe. On vérifie si ce trait nous a permis d'isoler un de nous échantillons.



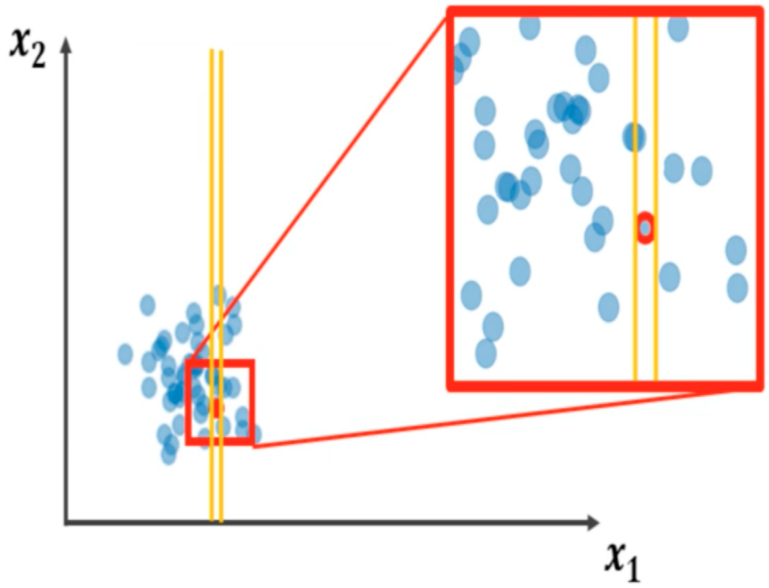
Est-ce que ce n'est pas une chance ?

Est-ce que ce n'est pas une chance ?



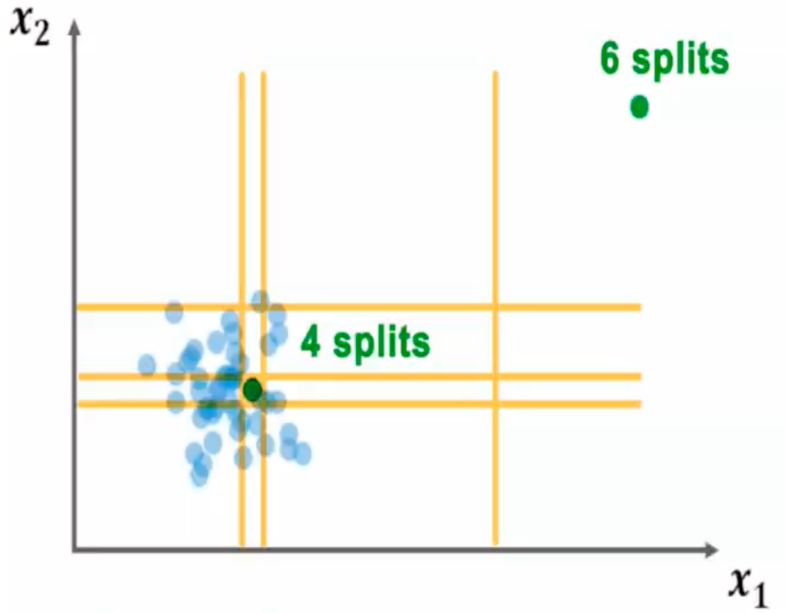
Plus de 1 chance sur 2 d'isoler d'isoler cet échantillons sur le domaine x_1.

Est-ce que ce n'est pas une chance ?



La probabilité pour y parvenir avant d'isoler la grosse anomalie est beaucoup plus faible.

Est-ce que ce n'est pas une chance ?



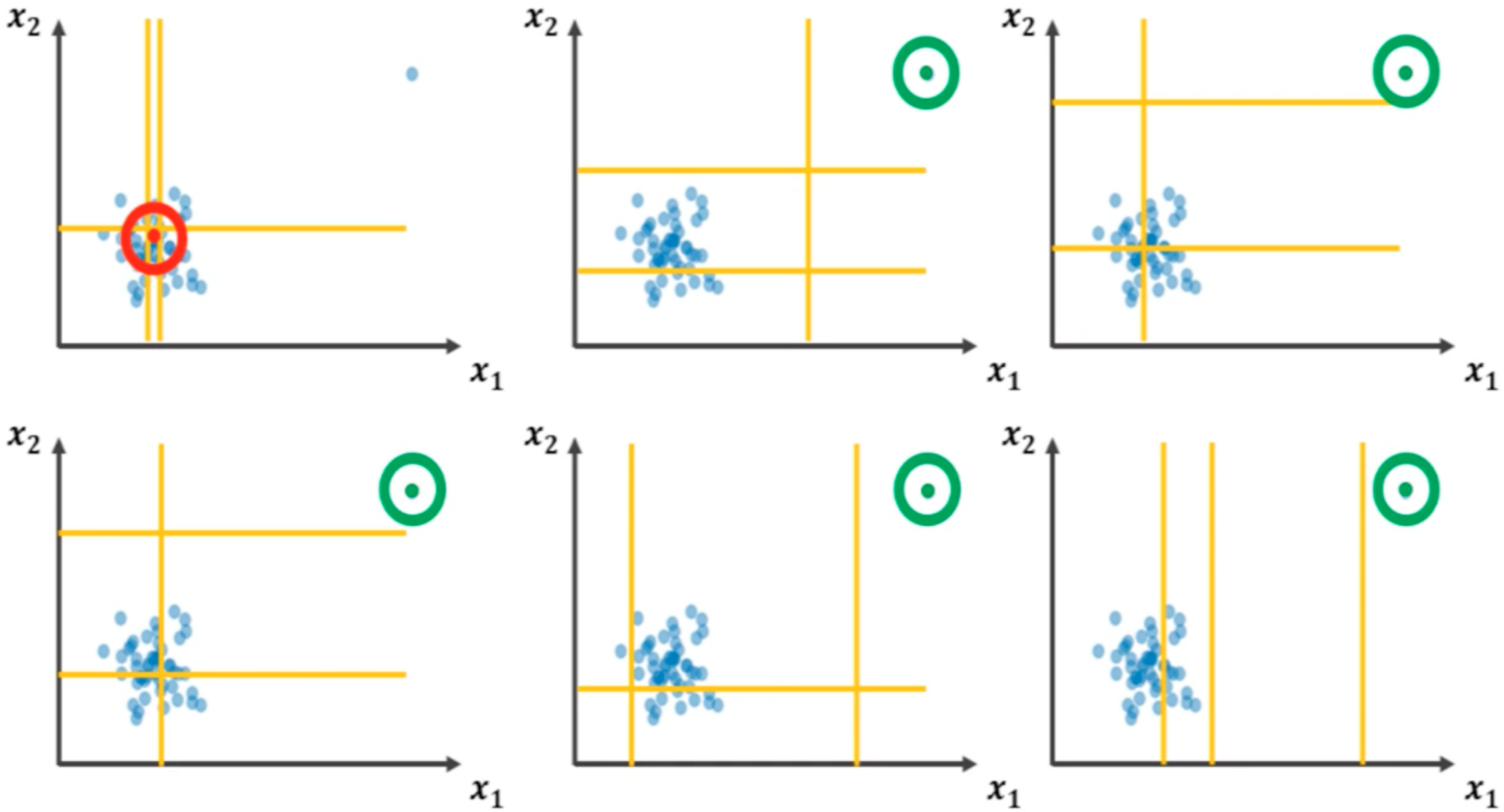
Problème : Même si la probabilité est faible, il est possible d'isoler un échantillons perdu dans la masse avant d'isoler la véritable anomalie.

Solution :

Technique d'ensemble

Demander à la machine de générer plusieurs estimateurs qui vont chacun effectuer une suite de splits aléatoires. En considérant l'ensemble de leurs résultats nous somme capable de disqualifier les erreurs qui peuvent être commises par certains estimateurs.

Solution :



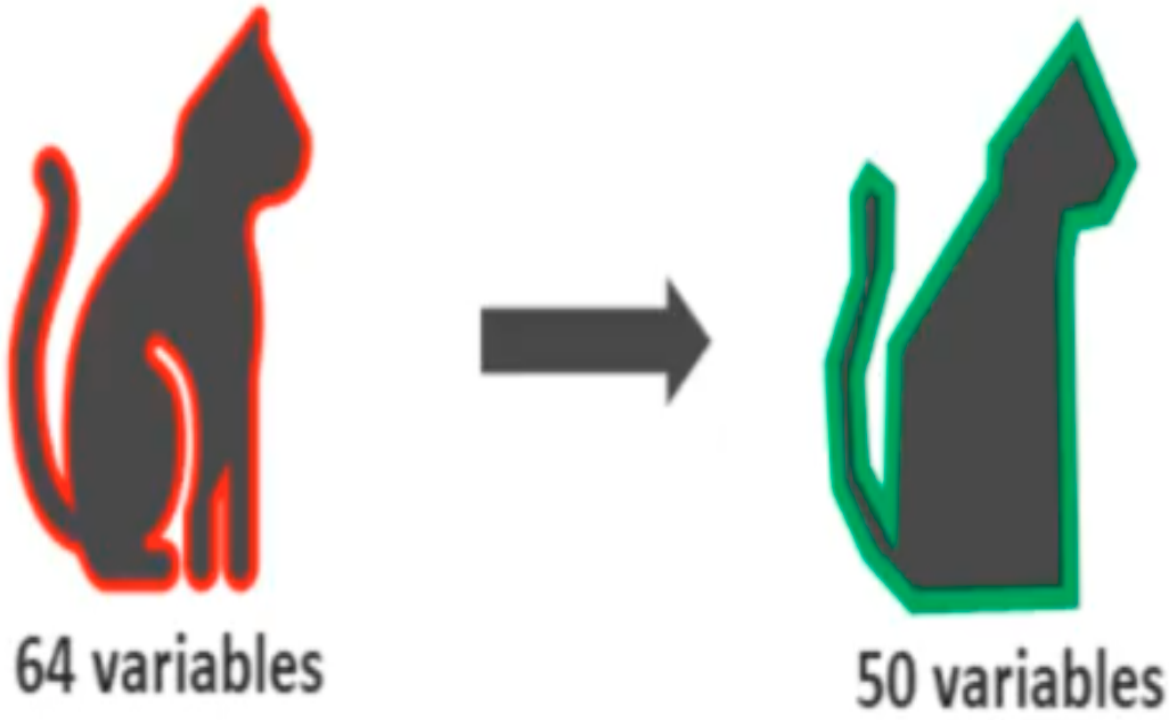
Réduction de Dimension

Analyse en Composantes Principales

Réduction de Dimension

Principe

Le principe est de réduire la complexité superflue d'un dataset en projetant ses données dans une espace de plus petite dimension.



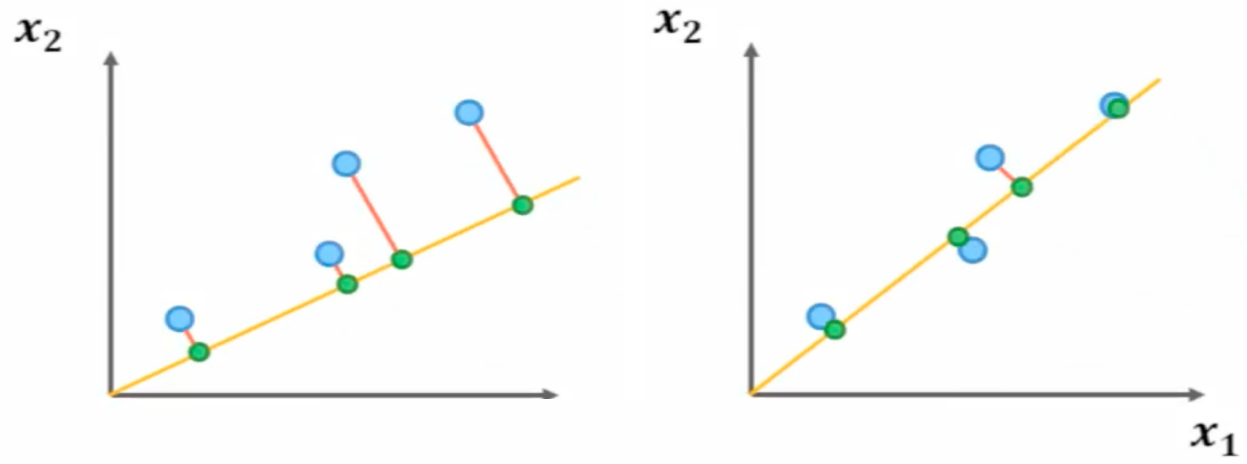
Réduction de Dimension

Objectif :

- Accélére r l'apprentissage de la machine.
- Lutter contre le phénomène appelé **fléau de la dimension** (risque d'overfitting lié surplus de la dimension)

Principe

Le principe est de projeter nos données sur des axes appelés **Composantes Principales**, en cherchant à minimiser la distance entre nos points et leurs projection.



```
1 from sklearn.decomposition import PCA
2 model = PCA(n_components = 10)
3 x_pca = model.transform(X)
```

⇒ **Comment choisir le nombre de composantes sur lesquelles projeter nos données ?**

Premier cas : Visualisation des données

On projette notre dataset sur un espace 2D ou 3D. (n_components =2 ou 3)

Deuxième cas : Compression de données :

Réduire au maximum la taille du dataset tout en conservant 95% - 99% de la variance des données.

On projette notre dataset sur un espace 2D ou 3D. (n_components =2 ou 3)

Réduction de Dimension

Plus on réduit la dimension du dataset, plus on perd en qualité des données.